FACULTY OF COMPUTING

SEMESTER 1 2025/2026

SECP3744 – ENTERPRISE SYSTEMS DESIGN AND MODELLING (WBL)

SECTION 1

TITLE:

ENTERPRISE SYSTEM DEVELOPMENT AND MANAGEMENT (ESDM) PROJECT

LECTURER: DR. ARYATI BINTI BAKRI

NAME: CHUA JIA LIN

MATRIC NO: A23CS0069

DATE: 10 JANUARY 2026

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

API	-	Application Programming Interface
EA	-	Enterprise Architecture
PTM	-	Pusat Teknologi Maklumat (Information Technology Centre)
SAP	-	Systems Applications and Products in Data Processing
TOGAF	-	The Open Group Architecture Framework
UI	-	User Interface
UTHM	-	Universiti Tun Hussein Onn Malaysia

LIST OF APPENDICES

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Universiti Tun Hussein Onn Malaysia (UTHM) is a public higher education institution that manages diverse academic and administrative activities to support students, academic staff, and university management. These functions are supported by several core operational information systems that collectively form the university's enterprise systems environment. As the scale and complexity of university operations increase, the need for structured coordination between business processes, information systems, and supporting technologies becomes more significant.

Currently, UTHM utilizes a basic form of enterprise architecture to support its information systems landscape. While this provides a general structure for system usage and management, it does not yet represent a comprehensive or formally defined enterprise architecture that fully aligns business needs with application, data, and technology layers. As a result, there is an opportunity to further develop and formalize the enterprise architecture to improve system integration, consistency, and long-term planning.

This project involves the development of a proposed enterprise architecture for UTHM that covers four core operational systems at the enterprise level. Each system is analyzed by a different project group to provide detailed architectural insights. The focus of this project is on Student Information System (SIS), with particular emphasis on the Student Course Registration subsystem, By applying enterprise architecture principles commonly used in higher education institutions, this project aims to propose a structured architectural model that supports clearer system alignment, improved information flow, and more effective management of student academic processes at UTHM.

1.2 PROBLEM STATEMENT

This section outlines the key problems that motivate the development of the proposed enterprise architecture for UTHM. It highlights current issues related to the existing enterprise architecture, associated inefficiencies, and potential risks affecting student-related systems. The discussion focuses on the Student Information System (SIS), particularly the Student Course Registration subsystem, to justify the need for a structured enterprise architecture approach.

1.2.1 Current Issues

UTHM utilizes multiple information systems to support its academic and administrative operations, including student related systems such as SIS. However, the enterprise architecture currently applied is basic and not formally documented as a comprehensive enterprise-wide model. As a result, the relationships between business processes, applications, data, and technology are not clearly defined across the university's core operational systems.

1.2.2 Inefficiencies

Due to the absence of a formalized and standardized enterprise architecture, coordination between systems may be inconsistent. System development and enhancement activities may be carried out independently by different units, leading to challenges in system integration and information flow. These inefficiencies are more apparent in student-related systems, where accurate data handling and process alignment are critical to support academic activities such as course registration.

1.2.3 Risks and Limitations

Without a clear architectural structure, the Student Course Registration subsystems under the SIS faces limitations in terms of scalability, and long-term planning. The lack of clear mapping between business requirements and system functions increases the risk of misalignment with UTHM broader academic and digital objectives. In addition, future system improvements may

become more complex and harder to manage without a structured enterprise architecture to guide decision making.

1.3 PROJECT OBJECTIVES

The main objective for this project is to design and purpose an enterprise architecture for Universiti Tun Hussein Onn Malaysia (UTHM). The objectives will be focused on analyzing the current system environment, applying a structured enterprise architecture framework which is TOGAF framework, and supporting future enhancing management of university operations. The objectives of this project are as follows:

1. To analyze the existing information system used by Universiti Tun Hussein Onn Malaysia (UTHM) including academic system, management system and others to gain greater understanding the current system environment and challenges.

2. To design an Enterprise Architecture (EA) for UTHM by using the TOGAF framework as a structured guideline to structure the business processes, applications, data, and technology.

3. To provide a structured architectural reference for future system improvement to make the system more manageable and systematic.

1.4 PROJECT SCOPE

This section defines the project boundaries to ensure that this project is achievable within the given constraints. It clearly outlines the areas that covered by the project (in-scope) and the part that are not included (out-of-scope) particularly related to the proposed enterprise architecture for UTHM.

1.4.1 IN-SCOPE

The in-scope areas of this project include:

- Analysis the existing enterprise systems environment at UTHM such as overview of academic, student, and management-related system.

- Focus on the Student Information System (SIS) specifically the student Course Registration subsystem as main case study.

- Design a proposed Enterprise Architecture for UTHM using the TOGAF framework as structured guideline that are covering key architectural layer such as Business, Application, Data and Technology Architecture.

- Create an architectural models and documentation as a reference for improvement of the system UTHM in future.

1.4.2 OUT-OF-SCOPE

The out-of-scope areas of this project include:

- Development a system, coding or any related to implementation of proposed enterprise architecture.

- Perform a testing such as security testing and user acceptance testing of existing or proposed system.

- Evaluation of hardware, cost analysis, or budgeting related to system implementation.

1.5 PROJECT IMPORTANCE

The development of a formalized Enterprise Architecture (EA) for UTHM is critical to ensuring the university's digital infrastructure can support its growing academic and operational demands. This project holds significant importance in three key areas:

1.5.1 Strategic Alignment and Resources Optimization

Currently, UTHM's information systems operate in silos, which often leads to redundant spending where different faculties might procure separate software solutions for similar needs. This project is important because it utilizes the TOGAF framework to create a unified blueprint. By centralizing the architecture, the university can identify overlapping resources and streamline IT investments. This ensures that every ringgit spent on technology aligns directly with UTHM's strategic objectives, reducing unnecessary costs, and fostering long-term sustainability.

1.5.2 Operational Efficiency and System Integration

The project addresses the critical issue of fragmentation within UTHM's current environment. By proposing a structured architecture for the Student Information System (SIS), this project

eliminates the inefficiencies caused by independent system development. It provides a standard reference for IT teams, ensuring that data flows seamlessly between the academic and administrative departments. This reduces data redundancy, minimizes the risk of integration errors, and simplifies the maintenance of complex university systems.

1.5.3 Enhanced Student Experience

Since the project focuses specifically on the Student Course Registration subsystem, its immediate importance lies in directly improving student experience. Course registration is a high-traffic process that frequently suffers from system congestion. A well-architected system ensures scalability, preventing system crashes during peak registration times. Furthermore, this architectural update aims to streamline the user interface, making the registration process more intuitive and less stressful for students.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

This chapter presents a literature review related to enterprise architecture (EA) in universities, providing both theoretical and practical foundation for this study. This chapter begins with EA definition, key concepts, and some applications within higher education institutions. Meutia et al. (2022) state that EA serves as a strategic framework that aligns business processes, information system, and technology infrastructure with organizational objectives, helping universities to manage complex operations and support digital transformation.

Besides, the review also highlights previous studies on EA in universities, emphasizing how EA frameworks have been implemented in universities to coordinate administrative, academic, and technical functions (Amin *et al.*, 2024). In addition, this chapter examines various subsystems, discussing the existing implementation and architectural considerations of these sub-systems. Furthermore, this chapter also explores the technologies used in EA and its sub-systems, including software platforms and architectural frameworks.

Finally, this chapter concludes with a summary of key findings from the previous studies to establish a solid foundation for understanding EA in universities worldwide.

2.2 ENTERPRISE ARCHITECTURE

Enterprise Architecture (EA) is a method that an organization uses to plan the business and IT in a way that each will fit together well. Following the TOGAF standard, EA segments the organization into different segments to show how processes, information, applications, and technology support each other (The Open Group, 2018). EA also serves as a clear blueprint to map business goals to their supporting IT systems (Lankhorst, 2017). EA typically includes four sectors which are Business Architecture, Data Architecture, Application Architecture, and Technology Architecture (The Open Group, 2018). Together, it helps everyone understand how the organization works and how IT supports the organization.

In University, the Data Architecture layer is crucial, as they require high data quality and data organization for all aspects of their business, including students, staff, and research. Previous works indicate that universities often use a TOGAF framework for guidance but they adjust it to their needs (Pedroso, 2021). Universities focus on management of organizational data, interoperability of organizational systems, accessibility and security. However, challenges like different departments managing their own systems, legacy and outdated technologies, and a lack of trained EA professionals prevent them from implementing EA effectively. Thus, universities require good planning and teamwork for the successful application of EA.

2.2.1 University of Lampung

A study by Nama, Tristiyanto and Kurniawan (2017) designed an adaptive IT infrastructure based on the enterprise architecture framework The Open Group Architecture Framework (TOGAF) Architecture Development Method (ADM) for the University of Lampung (Unila). The Academic Information System (Siakad) used by Unila, which has been operational for a long time, is not integrated with other applications such as finance, employee, and planning systems. This causes delays in gathering data when leaders or stakeholders request reports across the work unit.

Figure 2.1 shows the enterprise architect that was proposed to Unila. According to the study, the IT unit should be in charge of centralizing infrastructure management. In order to deliver a scalable, dependable, and flexible service, the unit must also implement security rules, data management and distributed system services and run the Disaster Recovery Center.

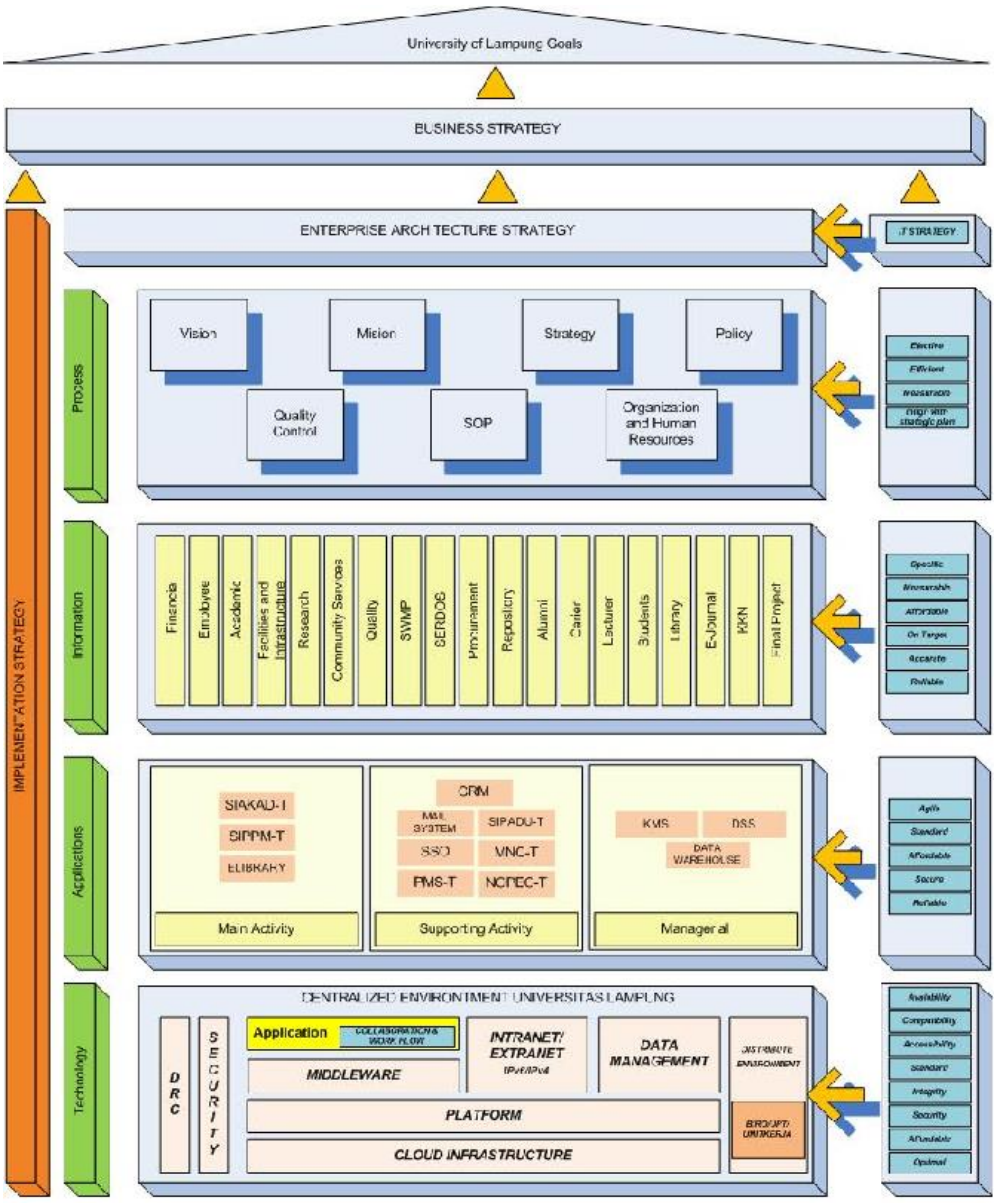


Figure 2.1 Enterprise Architecture proposed to Unila (Nama, Tristiyanto and Kurniawan, 2017)

2.2.2 NATIONAL UNIVERSITY SOUTH JAKARTA

Another study by Hindarto (2023a) investigates the importance of EA in enabling digital transformation of the management systems in universities. The paper uses The Open Group Architecture Framework (TOGAF) framework as its research method, as it provides a methodical framework for the development, organization, execution, and oversight of enterprise information technology architecture. The paper defined digital transformation as the migration from manual processes to using application systems that can automate those processes, as universities often face problems such as human errors and slow information flow and data retrieval from manual processes. The paper also highlights opportunities using application systems such as access to advanced technologies, artificial intelligence (AI), big data analytics, and the Internet of Things (IoT) that can increase effectiveness in operation, boost the process of teaching and use resources efficiently.

Figure 2.2 shows the enterprise architecture described by the paper. The Integration Services establishes a connection to multiple sources, allowing data integration from external systems using API. The portal is a central interface through which teachers, staff, and students can combine their access to information and university resources. Data & Analytics oversees collecting, analysing, and using data to better decision-making, while the Back-Office systems manage the internal operations of the university.

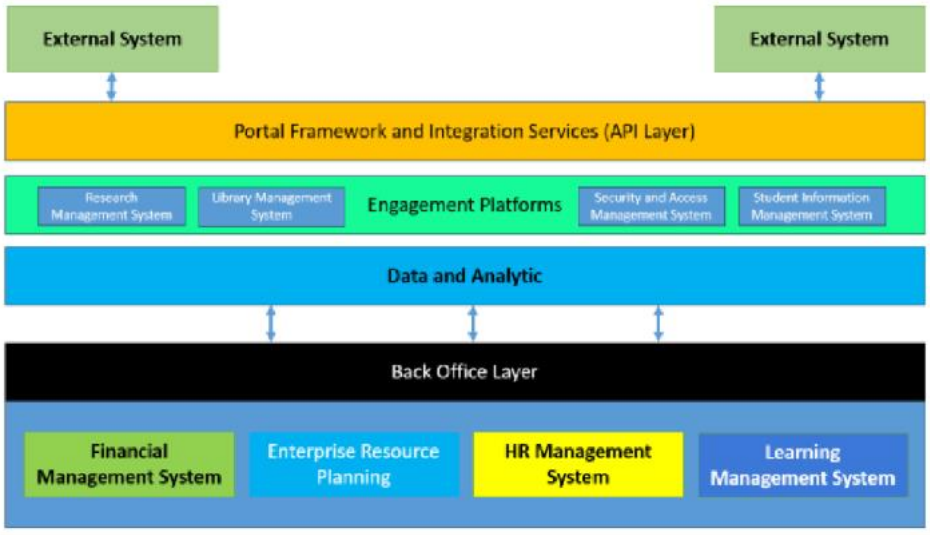


Figure 2.2 Enterprise Architecture by Hidarto (Hidarto, 2023a)

2.2.3 NEW YORK UNIVERSITY

Figure 2.3 shows the New York University's (NYU) Enterprise Architecture operating model where it explains how the university connects its overall strategy with IT planning and governance. NYU uses EA to connect key areas such as learning and teaching, research, community engagement, and administration with its IT and digital systems. This shows that EA is not only about technology, but it also about supporting the university's main goals. By aligning EA with NYU's strategic, the university ensures that IT investments and digital initiatives support teaching, research, and daily operations across different campus.

At the architecture level, NYU's EA covers business process, data, application and technology infrastructure. Important data such as student records, research information, and financial data are managed clearly to support reporting, better decision making, etc. Core system like the Student Information System (SIS), Learning Management System (LMS), and identity services are shared across the university to avoid any duplication in the system. In addition, technology standards such as cloud, security governance, and infrastructure guidelines help support both academic and research activity. Overall, New York University Enterprise Architecture shows how strong governance can help universities manage a complex system while keeping IT decisions match with institutional goals, making it useful reference for other universities.

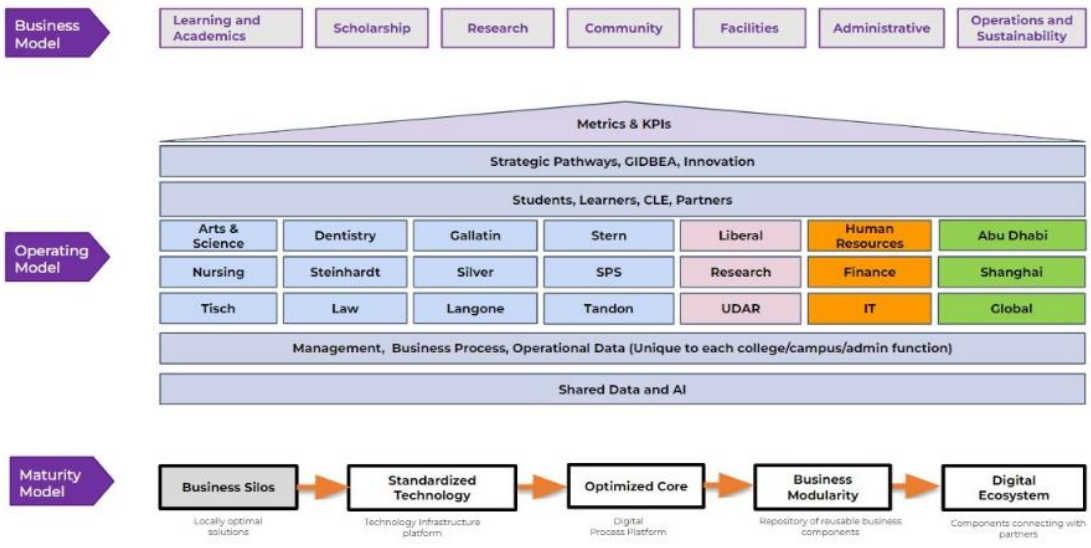


Figure 2.3 Enterprise Architecture of New York University (*NYU Operating Model Diagram*, no date)

2.3 RELATED SUB-SYSTEM

Research has been done to identify the subsystems used in university by analyzing them through the types of Enterprise Architecture methods and technologies. These studies look at the implementation and integration of systems like the Human Resources Management System (HRMS), Academic Information System (AIS), Student Information & Subject Registration System (SIS) and Learning Management System (LMS) and Business Intelligence (BI) and Data within EA frameworks to support coordinated processes, data consistency and higher education institutions' operational and strategic goals.

2.3.1 Human Resources Management System (HRMS)

An HRMS is a system that combines information technology and human resources management (HRM) to automate and support HR procedures. For instance, it manages payroll, training, performance reviews, resignations, resume (applicant), employee details and attendance. The goal is to increase operational efficiency in HR departments, decrease manual work and make data access more dependable and simpler (Navaz, 2013).

According to Setiawan (2018), the primary goal of HRMS is to guarantee that the organization's strategic goals can be met and the universities' needs can be satisfied in terms of both quantity and quality. The method used from the study is the Open Group Architecture Framework (TOGAF) 9.1 where it stated at preliminary phase; in addition to managing the standard data sources in universities like payroll, employee attendance, hiring and leave, HRMS also handles the demands of promotion management, period payment increase analysis, reward and punishment management and staff education. In particular, HRMS in universities mainly covers these functional areas including payroll management, human resources management, institutional management, civil service management, education and training management, performance management and function management. Furthermore, universities have a HRMS integrated system design that can be integrated with current systems so that its HRMS architecture may supply information required by end users as well as the business needs of universities in managing human resources.

2.3.2 Academic Information System (AIS)

A university's Academic Information System (AIS) is a software program that keeps track of all academic data including timetables, grades, courses and student information. Students can access their transcripts, study result cards and academic information online while administrators can effectively manage student records, enter grades, manage courses and create reports. It replaces manual, paper-based procedures, saves time, minimizes mistakes and improves staff and student access to academic management (Agnes, Jola and Gaspersz, 2018).

According to Astri and Gaol (2013), the academic information system manages the school's academic processes from new student entrance to graduation ceremonies. In the study, this system performs essential functions such as managing admission test results, scheduling, curriculum, grade management, student administration and academic report applications. The study uses the Enterprise Architecture Planning (EAP) methodology to connect technology, data and apps across the systems. Hence, better administration, coordination and long-term system planning are made possible by the school's ability to create a comprehensive blueprint through EAP that synchronizes its academic information systems with an integrated enterprise-level architecture.

2.3.3 Student Information & Subject Registration System (SIS) and Learning Management System (LMS)

The Student Information System (SIS) and the Learning Management System (LMS) constitute the core technological foundation that supports both academic administration and instructional delivery within a university. In contemporary digital transformation initiatives these systems are expected to function not as isolated components but as fully integrated parts of a comprehensive University Management System (UMS) (Hindarto, 2023a).

The SIS is responsible for handling the complete lifecycle of student information. This includes initial registration, enrollment in subjects and the maintenance of academic records, and

the management of graduation processes. Earlier forms of SIS frequently encountered issues such as fragmented datasets and duplicated entries which resulted in inconsistencies across institutional records (Lamey *et al.*, 2023). To address these challenges many institutions, rely on EA frameworks that are often modelled on methodologies such as TOGAF. These frameworks enable the design of more coherent processes, unified databases, and stronger linkages to subsystems that include finance and scheduling (Meutia *et al.*, 2022).

The LMS complements the SIS by managing instructional resources and assessments. Effective academic operations require that the LMS maintain seamless integration with the SIS to ensure the accurate synchronization of enrolment data and student performance records. Lamey et al. (2023) emphasize that the development of intelligent and integrated academic solutions depends on consistent and harmonized data shared across both systems. Case studies demonstrate that EA planning (Nama, Tristiyanto and Kurniawan, 2017) plays a central role in shaping the Application Architecture layers to support reliable academic data flows.

2.3.4 Business Intelligence (BI) and Data

The Business Intelligence (BI) and Data Subsystems represent the strategic dimension of Enterprise Architecture implementation. Its purpose is to transform institutional data into reliable insights that guide decision making, policy information, and overall governance. A central objective of EA within digital transformation initiatives is to advance universities toward a fully data driven operational environment (Hindarto, 2023a).

The effectiveness of any BI subsystem is determined by the strength of the Data Architecture established during the EA planning process. This stage involves the standardization of data definitions and the assurance of data quality throughout all primary source systems such as the SIS, HRMS, and FMS before these data are transferred into a centralized data warehouse (Meutia *et al.*, 2022). Without rigorous data governance the outputs produced by BI platform risk becoming fragmented or unreliable.

According to Lamey et al. (2023) advanced analytical and intelligent institutional systems require a well-integrated and dependable data foundation. Such a foundation allows universities to

perform tasks that include forecasting student retention, allocating resources efficiently, and monitoring institutional performance indicators. Consequently, the application of EA principles (Nama, Tristiyanto and Kurniawan, 2017) extends beyond merely connecting technological subsystems. It involves structuring the data environment in a way that supports meaningful strategic intelligence.

2.4 TECHNOLOGY USED

This section serves to describe and justify the key technological platforms, frameworks, and tools selected for the implementation of the enterprise system under study. This part highlights the infrastructure and digital solutions chosen for their relevance, effectiveness, or innovation, providing a clear rationale for why these technologies suit the project goals and context. In this case of university enterprise systems, this section explains how foundational technologies like SAP Business Technology Platform (BTP) support the digital transformation of campus services and operations, allowing for centralized management, real time analytics, and seamless integration across diverse academic and administrative functions.

2.4.1 King Abdullah University of Science and Technology (KAUST)

The project uses the SAP Business Technology Platform as a central technological enabler, similar to the real-life implementation at King Abdullah University of Science and Technology. Building on SAP BTP, KAUST created an integrated digital platform called "KAUST Central," which consolidated the essential campus services-dining, transportation, security, payments, and communication-into one mobile and web application. The system is exemplary, depicting how SAP BTP can enable a university-wide digital transformation through scalable cloud services, intelligent integration, and user-centered design.

Several SAP BTP components were leveraged for the KAUST project. SAP Mobile Services SDK was used to build native apps for iOS and Android, thus allowing real-time access to campus functions. SAP Analytics Cloud allowed data-driven decisions based on tracking student behavior and providing personalized content, while SAP Experience Management (Qualtrics) continuously gathered user feedback for refining campus services. Moreover, the SAP

BTP cloud infrastructure enabled seamless integrations with legacy systems and third-party applications through standardized APIs, which is very important in a distributed university setting where various departments operate on different platforms.

From an enterprise architecture perspective, with SAP BTP, KAUST has implemented a modular and flexible architecture scaling across departments while preserving centralized governance. Single sign-on simplified access to the system, while the cloud-native setup provided support for rapid updates and extensions, such as adding COVID-19 information features and smart city modules. Most impressively, the platform has managed to cut campus services' wait times from 10 minutes down to 40 seconds and allowed logistics, dining, and transport on campus to be coordinated through real-time dashboards and automated processes.

The use of SAP BTP at KAUST represents one way in which universities are moving beyond traditional ERP models toward a campus-as-a-platform model. It confirms SAP BTP as appropriate, future-ready technology for university enterprise systems, which supports digital innovation, integration, analytics, and smart campus ecosystems.

2.5 SUMMARY

The literature review highlights the importance of Enterprise Architecture (EA) in helping universities to align their business procedures, data management, applications, and technology infrastructure to meet institutional goals and digital transformation. Previous studies show that universities often adopt EA frameworks such as TOGAF framework, but universities usually adjust the framework so that they could fit their own needs, especially when dealing with issues such as decentralized departments, out-dated systems, and diverse stakeholder needs.

Case examples from New York University and University of Lampung demonstrate how EA can support better alignment between strategy and IT, improve data management, integrate applications, and set clear technology standards. Meanwhile, insights from the National University South Jakarta emphasize that universities often change from manual processes to digital transformation using application systems that can automate those processes.

In addition, the review of key university sub-systems, including HRMS, AIS, SIS, LMS, BI, and Data shows how EA helps universities ensure smooth data flow, reduce duplication, and design scalable systems. Furthermore, the use of technologies, such as SAP Business Technology Platform (BTP) as demonstrated by KAUST, further show how cloud-based platforms enable universities to deliver smart campus services and support long-term digital innovation.

In conclusion, the literature establishes EA as a strong foundation for modern university enterprise systems, providing both strategic direction and technology integration to enhance overall institutional performance.

CHAPTER 3
METHODOLOGY

3.1 INTRODUCTION

This project aims to design an enterprise architecture for Universiti Tun Hussein Onn Malaysia (UTHM). To achieve this goal, the Waterfall development approach is adopted in this project. The methodology guides the development process of this project from starting to the end, which is planning, analysis, design, implementation, and testing the enterprise architecture components. By following the methodology, the project ensures that every phase of the methodology was implemented successfully and aligned with UTHM operational requirements.

3.2 CHOSEN METHODOLOGY

The Waterfall methodology is a sequential development process where every phase must be completed before moving on to the next phase. It is suitable to be applied in projects that have clear and fixed requirements.

The reason why Waterfall methodology is chosen is because of its structured flow. Since enterprise architecture involves complex systems and long-term planning, understanding the correct requirements is very important. With this methodology, all requirements must be clearly understood before moving to next phase to design the real enterprise architecture.

Other than that, each phase in the Waterfall approach such as requirements analysis, design, implementation, and testing is clearly documented. These documentations will help the university a lot in future maintenance or enhancement.

Moreover, since the phases are clearly defined, it is easier to manage resources, manpower, and project schedules. This reduces uncertainty of the project and reduces the risk of project failure. Therefore, the Waterfall methodology is a suitable approach for this project.

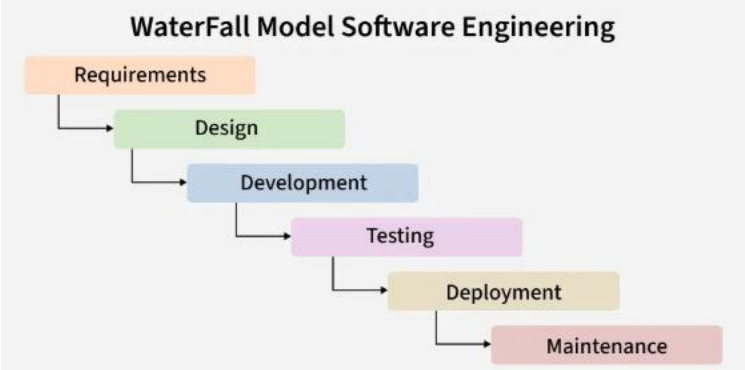


Figure 3.1 Waterfall Methodology Phases (GeeksforGeeks, 2026)

3.3 PHASES OF METHODOLOGY

As illustrated in Figure 3.1, this project is led by the six distinct phases of the Waterfall Model. Each phase depends on the deliverables of the previous one, making sure of a logical progression from recognizing the fragmentation issues at UTHM to delivering an interrelated Enterprise Architecture blueprint.

3.3.1 Requirements

The first phase involves gathering and analysing the necessary information to understand the current state of UTHM's business and IT environment specifically identifying pain points caused by the separation of various departmental systems. In this stage, the project team conducts data gathering to understand the existing siloed systems across different faculties and administrative units, defining the scope for unifying these departments into a single platform. The outcome of this phrase is a comprehensive Problem Statement and User Requirement Specification (URS) that clearly identifies the gaps in the current architecture.

3.3.2 Design

Based on the requirements gathered, the team moves to the design phase to create the architectural blueprints that define the "To-Be" state of the organization. The team adopts the TOGAF (The

Open Group Architecture Framework) approach to model the target architecture including the Business, Data, Application, and Technology layers required to support a unified university platform. This phase concludes with the production of high-level architectural diagrams and a proposed Enterprise Architecture design document that solves the issues identified in the analysis phase. technologies were defined in this phase in order to design the system architecture, which was done by using SAP Build Apps. Each member was assigned different modules and was assigned to make visual representations of the modules using UML diagrams such as use case diagrams, ERD diagrams, use case descriptions, activity diagrams, and sequence diagrams. Interface design was also provided to give an overview of how the finished product would look.

3.3.3 Development

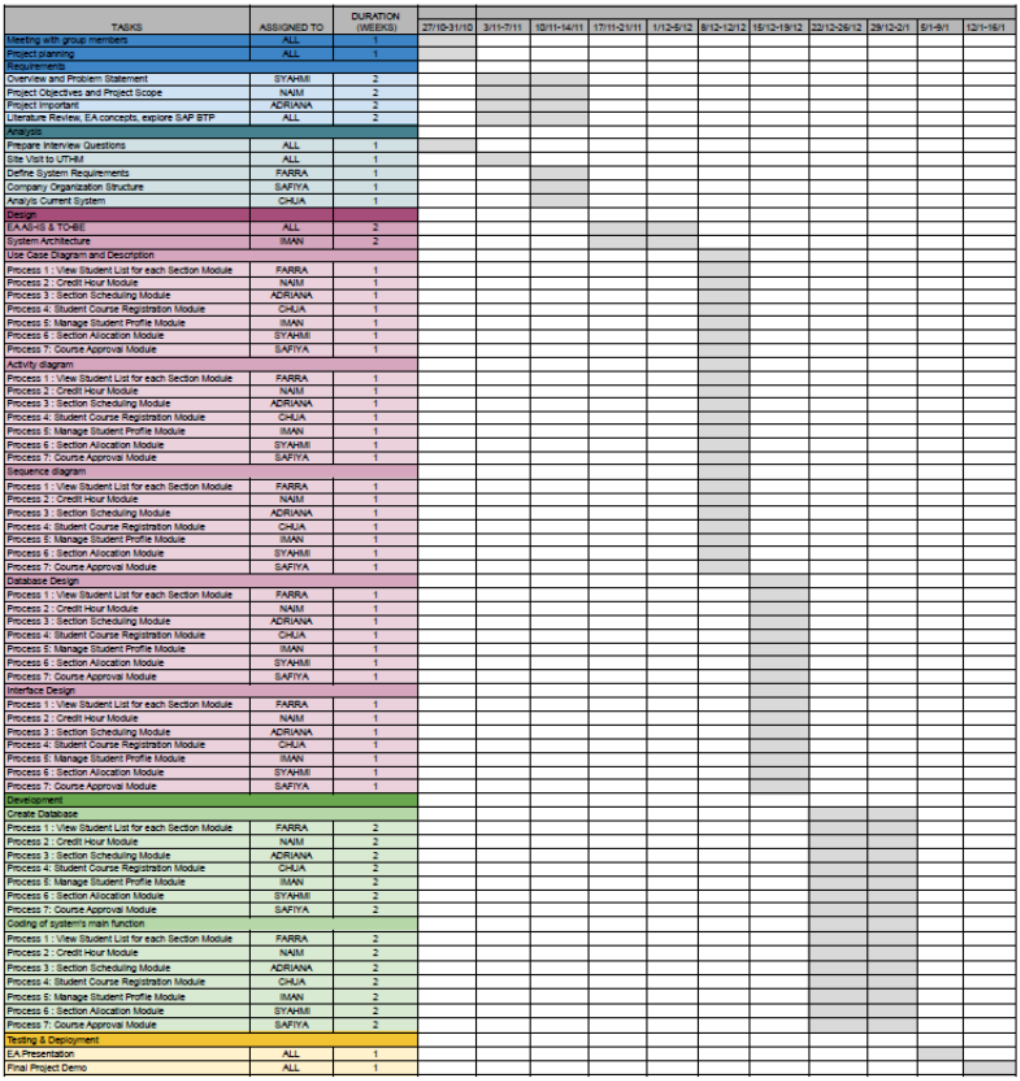
During the development phase, the system was constructed with SAP Build Apps, and the process began with a structured implementation of the data layer, building the necessary entities based on each member module. Once the data foundation was established, UI design and logic implementation were completed, with functional modules customized to each module.

3.3.4 Testing

This phase focused on testing the quality of the custom logic and data bindings, beginning with unit testing to ensure that the complex lookup formulae for dynamic session filtering produced exact data from the database. Functional testing was then performed using the SAP Build Apps Previewer, where each use case module was tested to ensure that the system appropriately handled both success and failure scenarios, such as mismatched passwords or wrong current credentials. Finally, integration testing verified that the data layer and user interface stayed in sync, guaranteeing that changes to one page of the system's information were promptly reflected in the application's local memory. The predicted and actual results were then compared to ensure that all functional requirements were properly met.

3.4 PROJECT PLANNING SCHEDULE

The project planning schedule, as illustrated in Figure xx, follows the sequential stages of the Waterfall methodology to ensure a structure development process. The timeline begins with a collective planning and analysis phase, where the entire team conducts a site visit to UTHM and prepares interview questions to define the system requirements. The project moves into design phase, where individual responsibilities are assigned to all team members to develop specific tasks for the seven modules. The schedule then transitions into the implementation phase, focusing on database creation, and coding. Finally, the process concludes with a testing and deployment phase.



TASKS	ASSIGNED TO	DURATION (WEEKS)	27/10-31/10	3/11-7/11	10/11-14/11	17/11-21/11	1/12-5/12	8/12-12/12	15/12-19/12	22/12-26/12	29/12-2/1	5/1-9/1	12/1-16/1
Meeting with group members	ALL	1											
Project planning	ALL	1											
Requirements													
Overview and Problem Statement	SYAHMI	2											
Project Objectives and Project Scope	NAIM	2											
Project Important	ADRIANA	2											
Literature Review, EA concepts, explore SAP BTP	ALL	2											
Analysis													
Prepare Interview Questions	ALL	1											
Site Visit to UTHM	ALL	1											
Define System Requirements	FARRA	1											
Company Organization Structure	SAFIYA	1											
Analyis Current System	CHUA	1											
Design													
EA AS-IS & TO-BE	ALL	2											
System Architecture	IMAN	2											
Use Case Diagram and Description													
Process 1 : View Student List for each Section Module	FARRA	1											
Process 2 : Credit Hour Module	NAIM	1											
Process 3 : Section Scheduling Module	ADRIANA	1											
Process 4: Student Course Registration Module	CHUA	1											
Process 5: Manage Student Profile Module	IMAN	1											
Process 6 : Section Alocation Module	SYAHMI	1											
Process 7: Course Approval Module	SAFIYA	1											
Activity diagram													
Process 1 : View Student List for each Section Module	FARRA	1											
Process 2 : Credit Hour Module	NAIM	1											
Process 3 : Section Scheduling Module	ADRIANA	1											
Process 4: Student Course Registration Module	CHUA	1											
Process 5: Manage Student Profile Module	IMAN	1											
Process 6 : Section Alocation Module	SYAHMI	1											
Process 7: Course Approval Module	SAFIYA	1											
Sequence diagram													
Process 1 : View Student List for each Section Module	FARRA	1											
Process 2 : Credit Hour Module	NAIM	1											
Process 3 : Section Scheduling Module	ADRIANA	1											
Process 4: Student Course Registration Module	CHUA	1											
Process 5: Manage Student Profile Module	IMAN	1											
Process 6 : Section Alocation Module	SYAHMI	1											
Process 7: Course Approval Module	SAFIYA	1											
Database Design													
Process 1 : View Student List for each Section Module	FARRA	1											
Process 2 : Credit Hour Module	NAIM	1											
Process 3 : Section Scheduling Module	ADRIANA	1											
Process 4: Student Course Registration Module	CHUA	1											
Process 5: Manage Student Profile Module	IMAN	1											
Process 6 : Section Alocation Module	SYAHMI	1											
Process 7: Course Approval Module	SAFIYA	1											
Interface Design													
Process 1 : View Student List for each Section Module	FARRA	1											
Process 2 : Credit Hour Module	NAIM	1											
Process 3 : Section Scheduling Module	ADRIANA	1											
Process 4: Student Course Registration Module	CHUA	1											
Process 5: Manage Student Profile Module	IMAN	1											
Process 6 : Section Alocation Module	SYAHMI	1											
Process 7: Course Approval Module	SAFIYA	1											
Development													
Create Database													
Process 1 : View Student List for each Section Module	FARRA	2											
Process 2 : Credit Hour Module	NAIM	2											
Process 3 : Section Scheduling Module	ADRIANA	2											
Process 4: Student Course Registration Module	CHUA	2											
Process 5: Manage Student Profile Module	IMAN	2											
Process 6 : Section Alocation Module	SYAHMI	2											
Process 7: Course Approval Module	SAFIYA	2											
Coding of system's main function													
Process 1 : View Student List for each Section Module	FARRA	2											
Process 2 : Credit Hour Module	NAIM	2											
Process 3 : Section Scheduling Module	ADRIANA	2											
Process 4: Student Course Registration Module	CHUA	2											
Process 5: Manage Student Profile Module	IMAN	2											
Process 6 : Section Alocation Module	SYAHMI	2											
Process 7: Course Approval Module	SAFIYA	2											
Testing & Deployment													
EA Presentation	ALL	1											
Final Project Demo	ALL	1											

Figure 3.2 Gantt Chart

3.5 SUMMARY

The enterprise architecture for Universiti Tun Hussein Onn Malaysia (UTHM) is developed using the Waterfall methodology, which follows a structured and sequential development process. This approach organizes the project into clear phases, including planning, analysis, design, implementation, and testing, ensuring that each stage is completed before moving forward and remains aligned with UTHM operational requirements.

The Waterfall methodology is suitable for this project because enterprise architecture involves complex systems, and long-term planning that require clearly defined requirements. Understanding all requirements at the early stage helps reduce the risk of system misalignment and improves the quality of architectural design. In addition, clear documentation produced in each phase supports future system maintenance and enhancement activities.

The project planning schedule follows the sequential flow of the Waterfall model, beginning with site visits and requirement analysis. The process then moves into the design phase, where responsibilities are distributed among team members, followed by implementation activities such as database creation and the use of framework such SAP Build. The project concludes with testing and deployment to ensure the proposed enterprise architecture functions as intended within the university environment.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 INTRODUCTION

This chapter's focuses on the design and analysis of the proposed system and the Enterprise Architecture (EA) for Universiti Tun Hussein Onn Malaysia (UTHM). It looks at the current system to find issues and requirements followed by the right enterprise architecture and system design approaches to create an improved EA. This chapter's outputs include the system architecture, database design and interface design to offer a clear foundation for the EA implementation stage.

4.2 COMPANY ORGANIZATION STRUCTURE

The Information Technology Centre (PTM) at Universiti Tun Hussein Onn Malaysia (UTHM) operates under a structured hierarchy led by a Director and a Senior Deputy Director, supported by an Office Secretary Assistant. The department is organized into six main divisions that handle different parts of the university's technology needs. The Application Management Division is responsible for the Application Development Unit and the Student and Staff Information System Unit. Infrastructure and Cyber Security handles data centre, networks, and telecommunications.

The Digitalization Technology, which handles web and multimedia services, and ICT Facilities and Support, which manages technical help and campus equipment. To ensure projects follow university rules, the Governance, Strategic, and ICT Training oversees planning and staff training. Finally, the Administration and Finance division manages the department's budget and office operations.

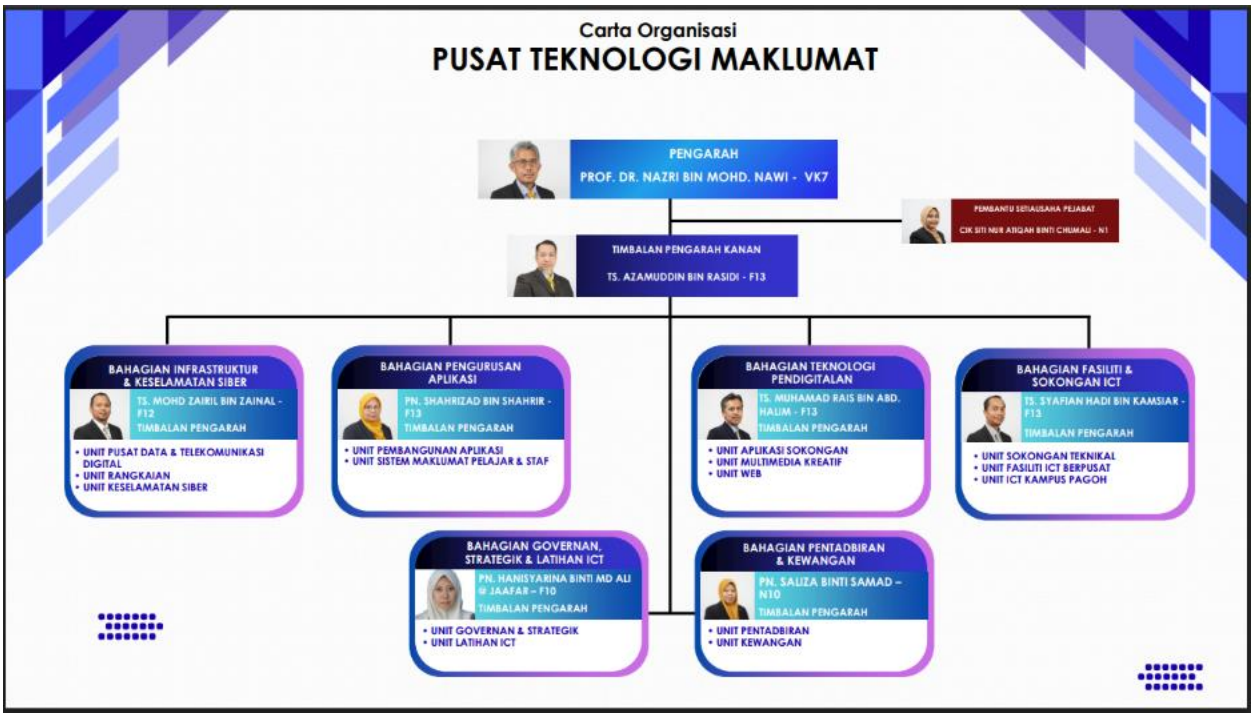


Figure 4.1 Information Technology Centre at UTHM (Administrator, no date a)

4.3 CURRENT SYSTEM ANALYSIS

UTHM currently uses a combination of limited manual processes and computerized systems. Their digital operations are supported by a Tier 3 infrastructure. The data center uses Data Structure Infrastructure Management (DCIM) monitoring, air-based condenser cooling system and hot and cold aisle containment to track performance and reduce power consumption. Additionally, they have a staging area where vendors set up systems before entering the containment area.

Furthermore, UTHM's primary digital technologies include a central student database and Learning Management System (LMS) for teaching materials. These systems are not fully integrated so the majority of data transfers are carried out manually one at a time especially when the format or structure varies between the systems. However, API connections are used to manage big data transfers to increase efficiency and lower errors. The process was also carried out for data replication to the Data Recovery Center (DRC) where data is transmitted for disaster recovery to a DRC in Pagoh. Moreover, Single Sign-On (SSO) was implemented to make system access easier but frequent user concerns like forgotten password and Wi-Fi login issues still happen.

Additionally, UTHM is transitioning from old systems to a new web-based integrated system called Total Campus Integrated System (TCIS). This migration is ongoing and being completed phase by phase based on modules. Even though UTHM wants to be entirely digital by 2030, some administrative tasks are still done manually because of outdated equipment and legacy systems. They stated that some hardware comes with outdated operating systems which makes it less compatible with modern technologies. In addition, UTHM uses a WAF (Web Application Firewall) to secure web portals and websites as well as two firewalls which filter threats often from China and another from Europe to protect the system and data.

Overall, UTHM is actively pursuing digital transformation and has built a strong infrastructure. However, system integration, manual data handling, legacy equipment and user access continue to be challenges in achieving full end-to-end digital transformation by 2030.

4.4 SYSTEM REQUIREMENTS GATHERING

System requirements gathering is the process by which we gather data to understand user needs and the enterprise system that will be developed. It helps us to determine what functions, features and improvements are needed. Two methods were used to collect data for this industrial visit which are questionnaire and interview. Before visiting UTHM, a set of Enterprise Architecture related questions was prepared based on the TOGAF elements. Moreover, we asked the presenter directly during the talk session in order to gather more information. Below are the sample questions that were presented using the TOGAF elements:

1. Business Architecture

- What are the university's primary business capabilities (e.g., teaching, research, finance, HR)?
- What is the top 10 business processes that require transformation?
- Which processes are manual, duplicated, or inefficient in the current environment?
- What future state vision or strategic goals does UTHM want the new EA to support?
- What are the critical KPIs (student success, research productivity, operational efficiency)?

2. Data Architecture

- What core data entities exist (students, staff, courses, finance, research, facilities)?
- What data standards or naming conventions does UTHM currently use?
- Where is data duplicated across systems?
- Is there single master database in UTHM for student data or separate with student database and learning database?
- What current workflow transferring between student data to LMS from the master database?

3. Application Architecture

- What are the current enterprise systems in use (student system, finance system, LMS)
- What functions do these systems provide today?
- What functions are missing?
- What is the most common complain receive from student for current online portal?
- What is the desired future application landscape (modular, unified, cloud-based)?

4. Technology Architecture

- What is the current hosting model (on-premises, cloud, hybrid)?
- Which operating systems, databases, and middleware solutions are used?
- What hardware or infrastructure is aging or needs replacement?
- What is the preferred technology stack for the new system?
- What precaution steps will be taken if electric is cut off?

5. Integration / Inoperability Architecture

- What systems need to integrate with each other (student system ↔ LMS ↔ finance)?
- What integrations currently exist, and what issues do they have?
- What is the strategy for data migration?

6. Security Architecture

- What are the authentication requirements (SSO, MFA, LDAP/Active Directory)?

7. Information / Content Architecture

- How are files stored today (file servers, Google Drive, departmental systems)?

8. Infrastructure Architecture

- How many data centres does UTHM operate?
- What backup and recovery processes are in place?

9. Governance Architecture

- Who makes decisions about IT investments and architectures?
- Does the university need EA governance tools (e.g., architecture repository)?
- Who will be responsible for reviewing or approving the system?

10. Change Management & Implementation Architecture

- How ready are stakeholders for digital transformation?
- What departments are most resistant to change?

4.5 SYSTEM DESIGN

This section describes the design of the proposed system based on the requirements identified during the analysis phase. The system design is presented using enterprise architecture models, system architecture, database design and interface design to clearly illustrate how the system is structured, user-friendly and capable of improving the efficiency of the existing processes at UTHM.

4.5.1 ENTERPRISE ARCHITECTURE EA – AS IS

Figure 4.2 illustrates the AS-IS enterprise architecture of UTHM. The figure was designed based on the requirements gathered during an interview session with their representatives. The architecture was designed according to their business goals, limitations and constraints specified by them during the interview. Each component of the EA will be discussed on the following sub chapters.

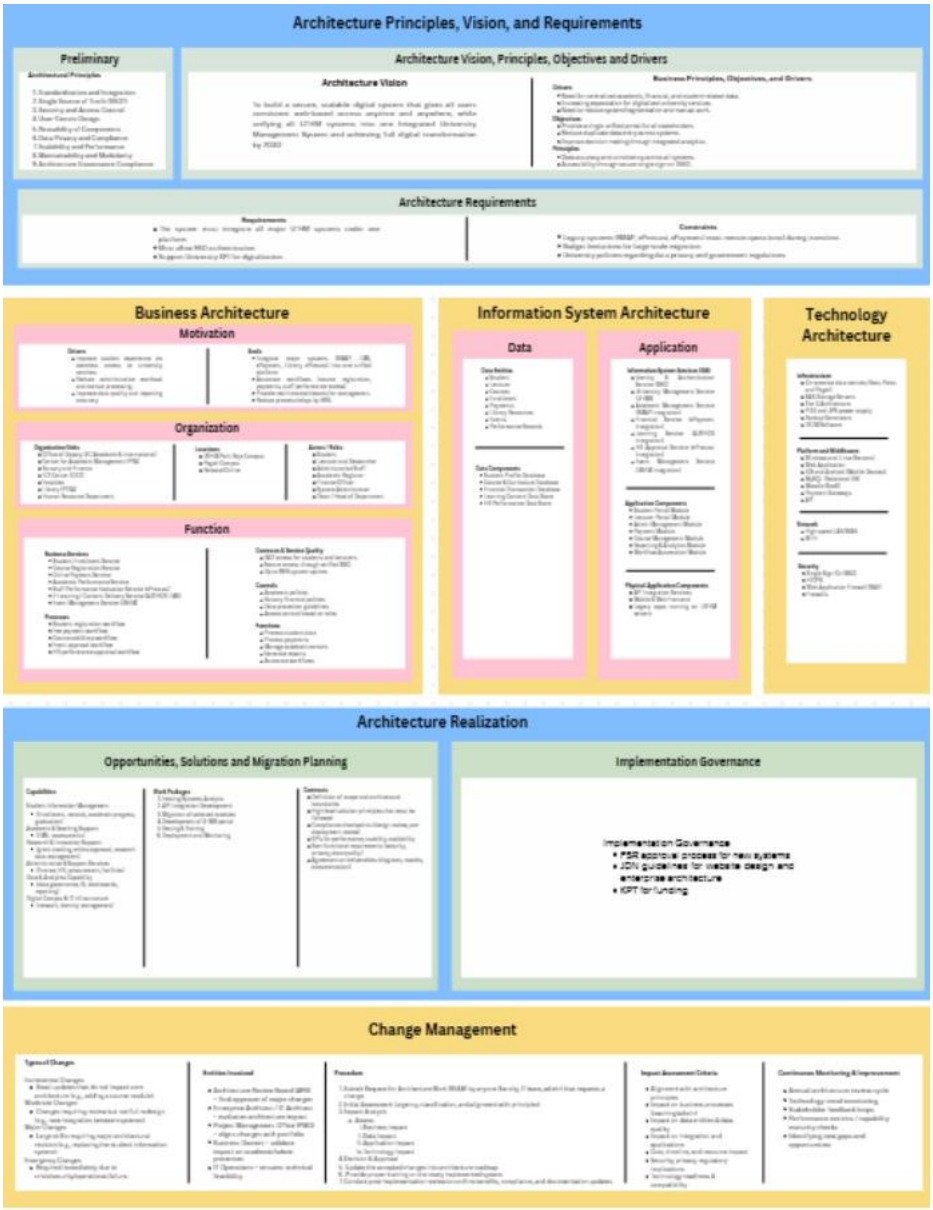


Figure 4.2 AS-IS Enterprise Architecture of UTHM

4.5.1.1 EA – AS IS (Architecture Principles, Vision, and Requirements)

Figure 4.3 shows the diagram that illustrating the architectural principles, visions and requirements. The primary goal of this design is to accomplish complete digital transformation by 2030. This is built on a goal to create a secure, scalable, and unified Integrated University Management System that offers all users uniform web-based access from anywhere. Several business reasons are driving this change, the most important of which is the necessity to combine distributed academic and financial data, as well as the growing demand for digital services. UTHM hopes to provide a single interface for all stakeholders, minimizing manual effort and boosting decision-making through integrated analytics.

To ensure that the system has a strong structure, the preliminary phase develops nine basic architectural concepts. These serve as the "base rules" for development, stressing standardization and integration, as well as a single source of truth (SSOT) to avoid data silos. The principles also focus the user experience through user-centric design while ensuring long-term sustainability through maintainability, modularity, and reusability. Security is a non-negotiable component, with a particular emphasis on data privacy, compliance, and access control, to ensure the architecture stays controlled and legally sound.

The current Architecture Vision (AS-IS) is primarily driven by operational stability and department independence rather than a unified enterprise strategy. It focuses on maintaining the functionality of distinct legacy systems like SMAP and ePayment to ensure daily academic and administrative continuity, relying on established on-premises infrastructure and manual data synchronization to meet immediate, localized business needs without disrupting ongoing university operations.

The effective execution of this vision is dependent on achieving architecture requirements, such as integrating all key existing UTHM systems and implementing Single Sign-On (SSO) for smooth user authentication. However, these objectives must be balanced against real-world constraints. Developers must guarantee that essential legacy systems, such as SMAP, ePrestasi, and ePayment, remain fully functioning during the transition phase. Furthermore, the change must be completed within the constraints of the university's budget and tight government data protection requirements.

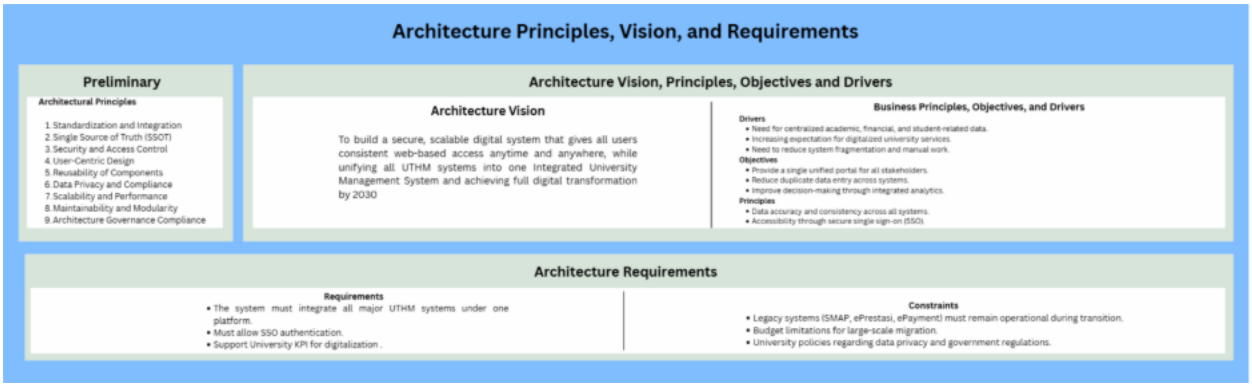


Figure 4.3 EA AS-IS (Architecture Principles, Vision, and Requirements)

4.5.1.2 EA – AS IS (Business Architecture, Information System Architecture, Technology Architecture)

Figure 4.4 illustrates the business architecture, information systems architecture and the technological architecture in the EA. The business architecture layer is the foundation, outlining the motivation, organization, and function of the university's digital transformation. It is driven by its goal to improve student experiences while reducing administrative costs by integrating significant systems such as SMAP and LMS. An important performance aim is to cut process delays by 50% while maintaining 99% system uptime. This layer distinguishes between different organizational units, such as the ICT Center and the Bursary, and describes the responsibilities of individuals ranging from students and lecturers to system administrators. Functionally, it defines basic services such as student enrollment and fee collection, which are guided by academic and financial rules to ensure high standards of service.

This information system architecture defines the gap between business needs and technological execution by focusing on data and application components. Data architecture specifies the important entities such as students, lecturers and courses, in addition to the underlying data components, such as the student profile and financial transaction databases, which contain this information. The application architecture describes the Information System Services (ISS), which include the University Management Service (U-IMS) and particular modules for students and teachers. It focuses on the integration of historical systems (such as ePayment and ePrestasi)

via an API integration layer, guaranteeing that disparate software components may efficiently communicate under a single site.

The Technology Architecture defines the physical and logical infrastructure needed to support the above levels. The infrastructure is created on-premises data centers in Batu Pahat and Pagoh, with Tier 3 architecture and SAN storage servers to provide high availability. The Platform and Middleware section includes information about Windows and Linux platforms, MySQL databases, and iOS and Android mobile compatibility. This complete stack is linked by high-speed LAN/WAN and Wi-Fi networks. The design protects university data with powerful security methods such as Web Application Firewalls (WAF), HTTPS encryption, and Single Sign-On (SSO).

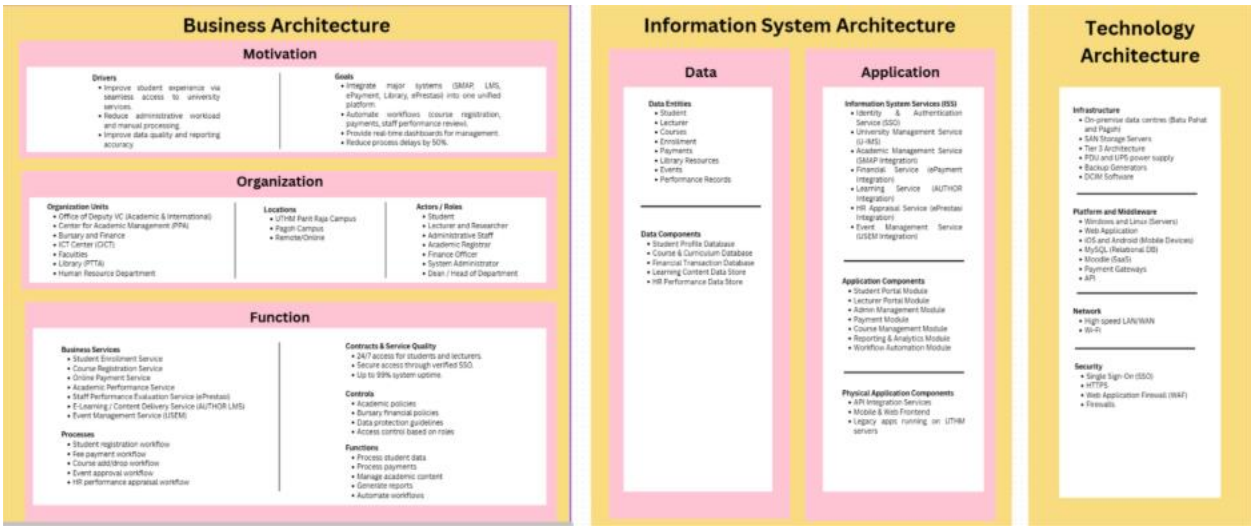


Figure 4.4 EA AS-IS (Business Architecture, Information System Architecture, Technology Architecture)

4.5.1.3 EA – AS IS (Architecture Realization)

The below EA AS-IS diagram shows the current situation of UTHM's enterprise systems. At present, UTHM already has important capabilities such as Student Information Management, Academic Support, Administrative Services, data & Analytics, and IT Infrastructure. However, many of these systems still work separately. Some processes depend on manual work, and system integration is limited. Because of this, data sharing between systems can be slow, and it is harder

for the university to monitor performance and make quick decisions. This situation creates an opportunity to improve efficiency and reduce system duplication.

From the migration planning perspective, the AI-IS stage focuses on basic analysis and preparation. The work packages mainly involve existing system analysis, initial API integration, and testing. These steps help UTHM understand which systems need improvement and how they can be connected. The contracts section highlights the importance of defining system boundaries, security requirements, and performance indicators before any major changes are made. Overall, the AS-IS architecture helps UTHM identify weaknesses in the current environment and prepares the university for a more structured transformation.

As shown in Figure 4.5, the Implementation Governance framework for UTHM's current enterprise architecture is defined by a structured oversight mechanism designed to ensure that all technical developments align with both internal policies and national standards. The governance process is anchored by three primary pillars which are the FSR approval process for the validation of new systems, the JDN (Jabatan Digital Negara) guidelines which standardize website design and enterprise architecture practices, and KPT (Ministry of Higher Education) for the strategic oversight of project funding. This framework ensures that any architectural changes, whether incremental or major, remain compliant with institutional mandates while maintaining the operational integrity of critical legacy systems.

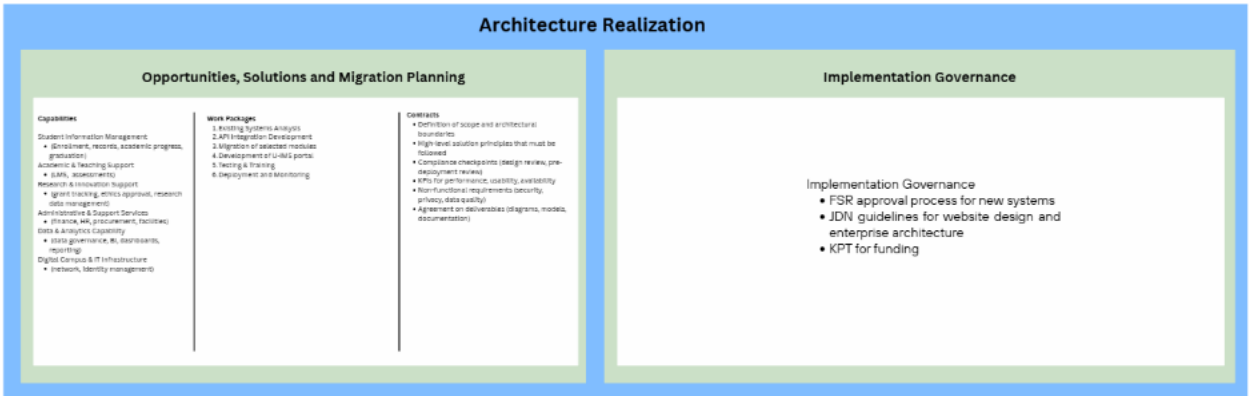


Figure 4.5 EA AS-IS (Architecture Realization)

4.5.1.4 EA – AS IS (Change Management)

Figure 4.6 shows the change management component within the EA. The Architecture Change Management (Phase H) creates a governance structure to guarantee that the university's digital systems change in an organized way rather than through random changes. It divides modifications into three categories: process simplification, gradual process incrementation, and system redesign. This guarantees that minor enhancements are implemented quickly, while high-impact changes are examined with extensive evaluation required to preserve system reliability and security.

The Architecture Review Board (ARB) manages the process, which is started by a formal change request if there are new technologies or changes in business needs. Each proposal must go through an extensive impact analysis to determine its impact on data integrity, current integrations, and the user experience. This phase ensures that all technological evolutions stay firmly connected with the university's long-term strategic strategy for 2030 by requiring official permission prior to execution.

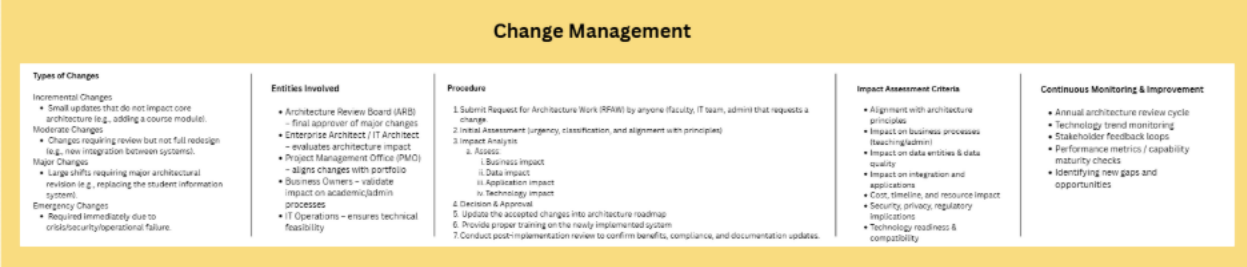


Figure 4.6 EA AS-IS (Change Management)

4.5.2 ENTERPRISE ARCHITECTURE EA – TO BE

Figure 4.7 below shows the proposed TO-BE enterprise architecture. It serves as the blueprint for UTHM on the proposed recommendations that can be improved based on the AS-IS enterprise architecture that still aligns with their business while at the same time improving efficiency. Some components have changes based on recommendations to improve business model while some principles remain the same.

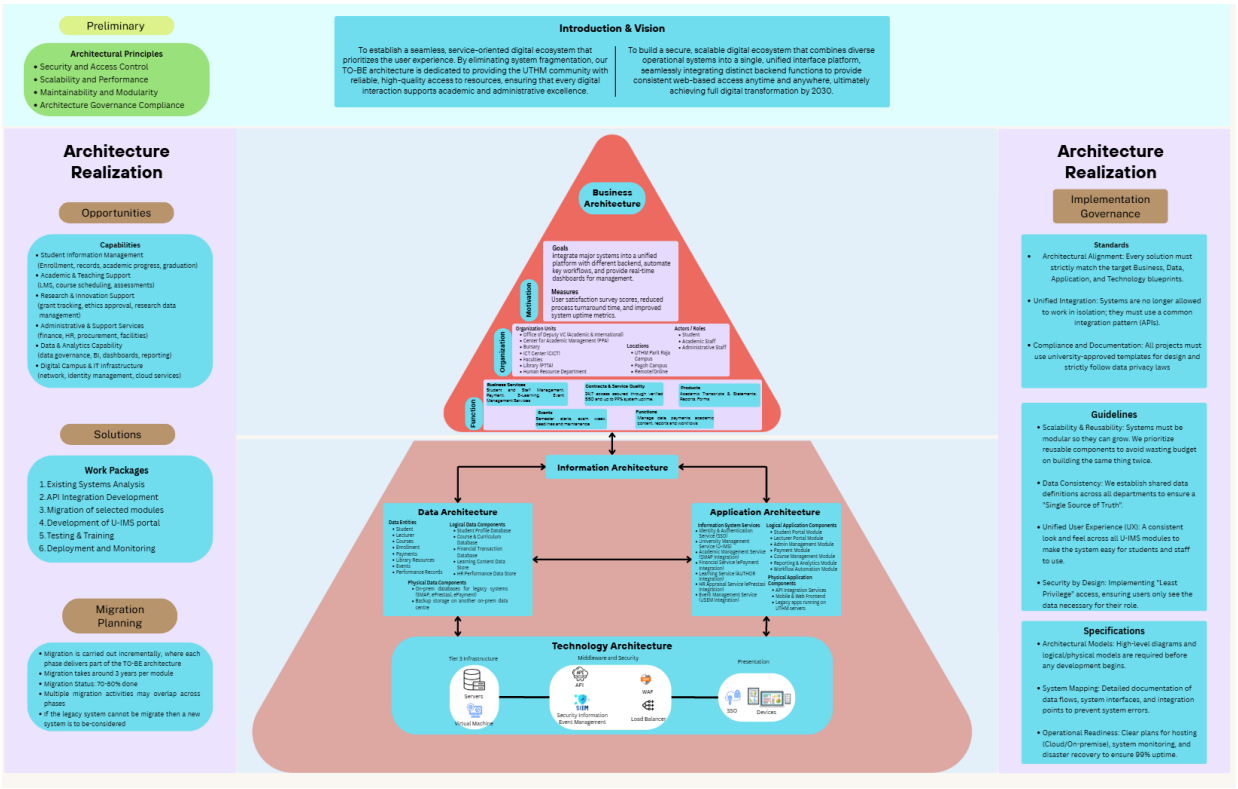


Figure 4.7 TO-BE Enterprise Architecture of UTHM

4.5.2.1 EA – TO BE (Architecture Principles, Vision, and Requirements)

Figure 4.8 shows the diagram that illustrating the architectural principles, visions and requirements. The primary goal of this design is to accomplish complete digital transformation by 2030. This is built on a goal to create a secure, scalable, and unified Integrated University Management System that offers all users uniform web-based access from anywhere. Several business reasons are driving this change, the most important of which is the necessity to combine distributed academic and financial data, as well as the growing demand for digital services. UTHM hopes to provide a single interface for all stakeholders, minimizing manual effort and boosting decision-making through integrated analytics.

To ensure that the system has a strong structure, the preliminary phase develops four basic architectural concepts. These serve as the "base rules" for development, stressing scalability and modularity. The principles also focus on architecture governance compliance so that all systems follow the architecture planned. Besides, security is a non-negotiable component, with a particular emphasis on data privacy, compliance, and access control, to ensure the architecture stays controlled and legally sound.

The proposed Architecture Vision (TO-BE) aims to realize UTHM's Digital Transformation 2030 agenda by establishing a unified, cloud-enabled Smart Campus ecosystem. This vision shifts the strategic focus towards high interoperability and a Single Source of Truth (SSOT), leveraging technologies like SAP Build Apps and SIEM to create a semaless, secure, and user-centric environment that integrates all academic and management functions functions into a single, scalable platform.

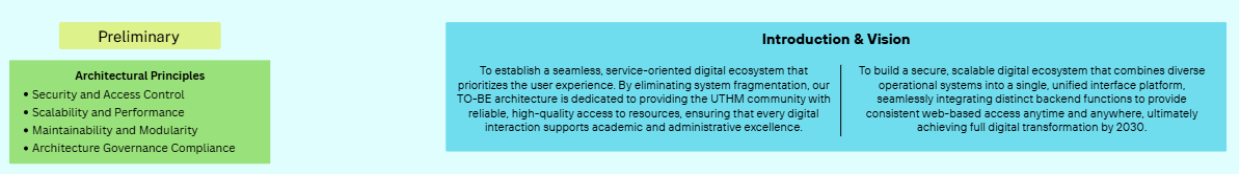


Figure 4.8 EA TO-BE (Architecture Principles, Vision, and Requirements)

4.5.2.2 EA – TO BE (Business Architecture, Information System Architecture, Technology Architecture)

Figure 4.9 below shows the TO-BE business architecture, information architecture, and technology architecture.

The business architecture TO-BE remains the same as the business architecture AS-IS where it still focuses on improving the student experience and reducing the administrative workload. The architecture still involves the same organizational units and roles including academic, ICT, and administrative departments. In addition, the core function and services remain the same, such as Student Enrollment Service, Course Registration Service, Online Payment Service, and Academic Performance Service. This consistency that the existing business architecture is harmonized well with university's objective where it does not require a significant change in organizational structure or functionalities.

The TO-BE information system architecture represents a better future state than the AS-IS by moving from basic, siloed systems to an integrated, service-oriented environment. The TO-BE architecture centralizes and increases data entities such as enrollment, payments and performance records and adds standardized services like single sign-on and system integration which differ to the AS-IS architecture which primarily covers basic tasks with minimal integration. This change makes it possible for modular applications, enhanced security, improved interoperability, and increased scalability to meet academic and business requirements.

The changes from AS-IS to TO-BE technological design propose an increase in system durability, data flexibility, and continuous security monitoring. While the basic infrastructure remains focused on Tier 3 on-premises data centers in Batu Pahat and Pagoh, the TO-BE architecture incorporates offsite recovery to assure business continuity and data availability during local disasters, which is a significant improvement over the present system. Furthermore, the usage of VMware for virtualization results in a more resilient and scalable hosting environment than the present basic web application architecture. The network and security layers get major performance and monitoring improvements. The TO-BE architecture includes load balancers to manage large traffic levels. Most significantly, the security infrastructure is improved by the implementation of Security Information and Event Management (SIEM), which enables real-time threat detection and monitoring while supporting the university's current firewalls and Web Application Firewall (WAF).

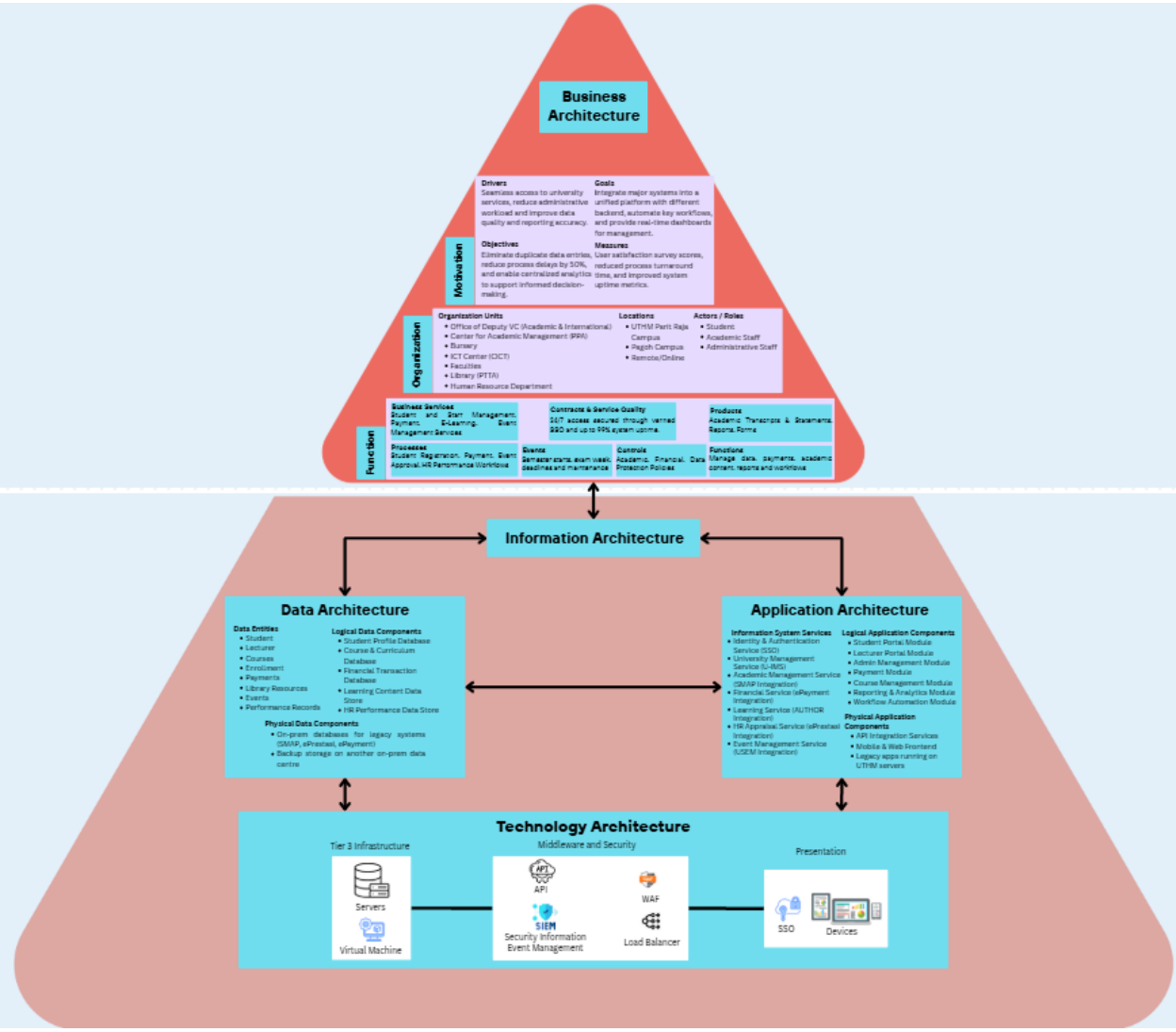


Figure 4.9 EA TO-BE (Business Architecture, Information System Architecture, Technology Architecture)

4.5.2.3 EA – TO BE (Architecture Realization)

Figure 4.10 below presents the future direction of UTHM's enterprise systems. In this architecture, the focus is on modernization, integration, and digitalization. New solutions such as API integration development, SAP BTP configuration, and a unified U-IMS portal are introduced. These solutions aim to improve system connectivity, reduce manual work, and provide better user experience for students and staff. Data is managed more centrally, which helps improve reporting, data accuracy, and decision-making.

For migration planning, the TO-BE architecture outlines a clearer and more structured roadmap. The work packages include module migration, system development, testing, training, deployment, and monitoring. This ensures that systems changes are done step by step without disrupting daily university operations. The contracts section becomes more detailed by including compliance checkpoints, non-functional requirements such as security and privacy, and clear deliverables. Overall, the below figure shows how UTHM can move from a partially integrated environment to a fully digital, secure, and well-managed enterprise system that supports long-term growth and digital transformation.

Figure 4.10 illustrates the Implementation of Governance for the university's target enterprise architecture. It is structured into a comprehensive framework consisting of three core pillars which are standards, guidelines, and specifications. The standards section establishes the mandatory rules that all new IT developments must satisfy. Architectural alignment ensures every solution fits the university's master plan, while Unified Integration mandates the use of APIs to break down system silos. Furthermore, Compliance and Documentation standards ensure all projects follow legal data privacy requirements and use standardized design templates.

Furthermore, the Guidelines provide best practices for high quality system design. Scalability and Reusability promote modular build to save costs, and Data Consistency ensures a "Single Source of Truth" across all departments. To improve accessibility, the Unified Use Experience (UX) mandates a consistent interface, while Security by Design limits data access to a "Least Privilege" basis. Additionally, the specifications act as the technical blueprints required before launching. Architectural Models provide the visual structure of the system, while System Mapping documents precise data flows and integration points to prevent errors. Finally, Operational Readiness ensures that hosting, monitoring, and disaster recovery plans are in place to support the university's 99% uptime goal.

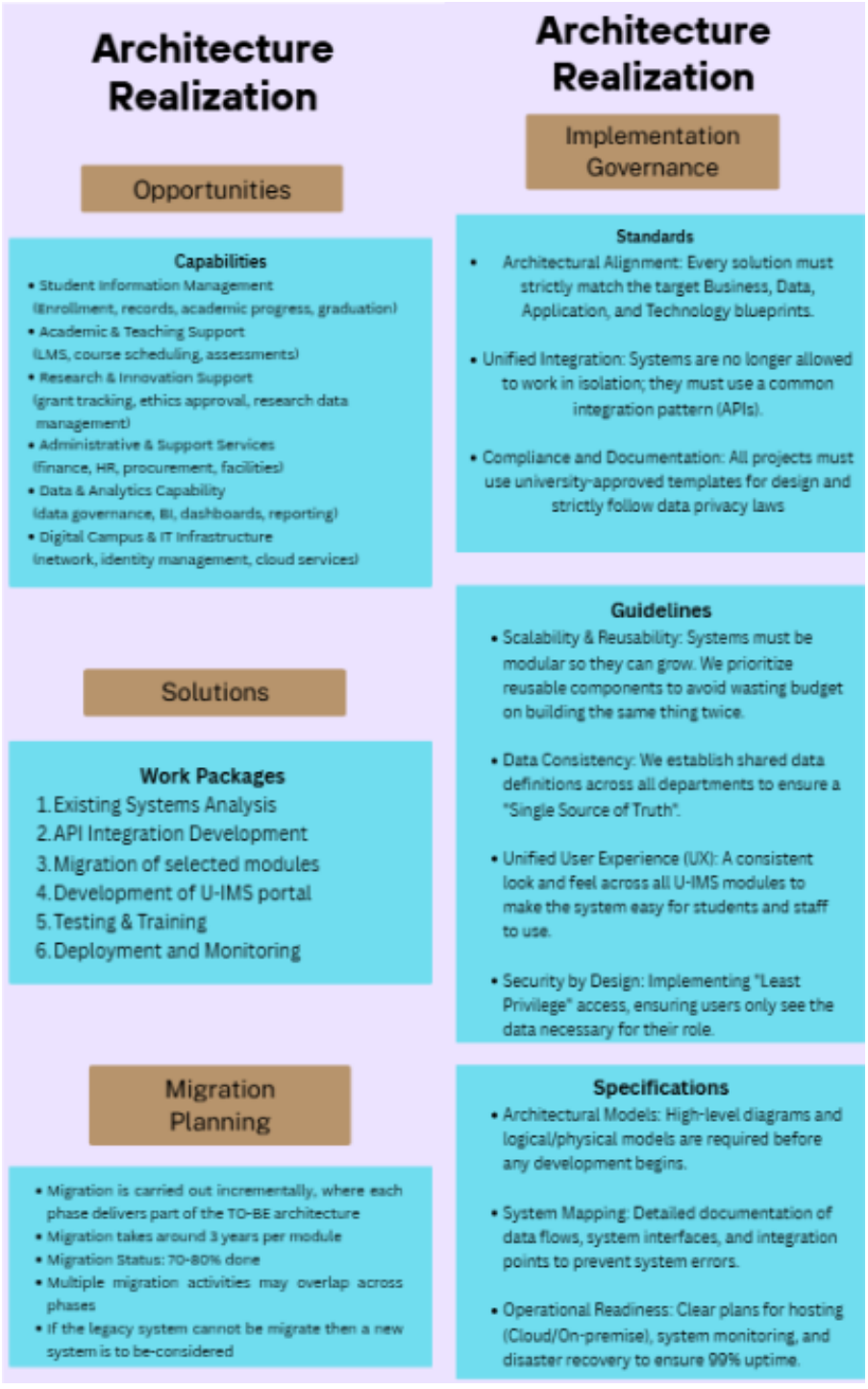


Figure 4.10 EA TO-BE (Architecture Realization)

4.5.3 SYSTEM ARCHITECTURE

Figure 4.11 shows the cloud system architecture designed for the academic management platform. As the applications are developed using cloud platforms called SAP Build Apps and SAP Build Process Automation, a cloud-based architecture is used.

The academic management platform is hosted on SAP Cloud and built using SAP Build Apps and SAP Build Process Automation. The access to the system is provided to students, lecturers, and administrators through the Internet via web or mobile devices.

The core services including course registration, course management, section scheduling, admin dashboard, course approval, student profile management, and viewing student lists. These services are executed in the SAP Build Apps Runtime and SAP Build Process Automation Runtime.

The data is stored and managed using cloud-based, in-memory, and on-device storage, such as the app variables, page variables, and SAP Cloud databases to ensure fast and reliable access. The system access is protected through user authentication and role-based access control, while the system performance is monitored through application monitoring, user management functions and logging.

Overall, a secure and automated cloud-based solution for academic management is provided through this architecture.

Figure 4.11 Cloud System Architecture for the System

4.5.4 PROJECT DESIGN

The design of the Student Course Registration subsystem was focused on this section. Object-oriented modelling tools such as use case diagram, use case descriptions, activity diagrams, and sequence diagrams were used to explain the system structure and workflows.

4.5.4.1 Use Case Diagram for Student Course Registration System

Figure 4.12 shows the Use Case Diagram for the Student Course Registration System. Three main functions were implemented in the subsystem, which were the Add Course and Section, Delete Course and Section, and Submit Course Registration.

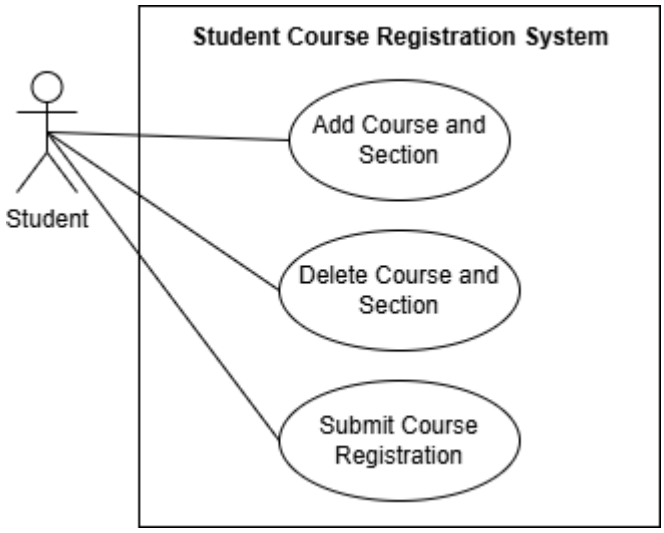


Figure 4.12 Use Case Diagram for Student Course Registration System

4.5.4.2 Use Case Description for Student Course Registration System

The Use Case Description for the functions in the Student Course Registration System were described in this section.

The Use Case Description for Add Course and Section module is shown in Table 4.1. This use case involves student as the user. The Add Course and Section process is started with course and section selection. A course and section must be selected from the dropdown list before the "Add Course" button is clicked. If a course and section was selected, a check is performed to determine whether the registeredCourse list contains course code of the selectedCourse list. If no course and section was selected, an alert with title "No Course and Section Selected" will be displayed. If the course code is not found in registeredCourse list, the selectedCourse is appended to the registeredCourse list and the value of selectedCourse is set to be empty. This means the selectedCourse is added and selectedCourse is ready to store another course and section. If the course code is already found in the registeredCourse list, an alert with title "Duplicated Registration for Same Course" is displayed.

The Use Case Description for Delete Course and Section module is shown in Table 4.2. This use case involves student as the user. The Delete Course and Section process is started with course code insertion. A course code must be entered before the "Delete Course" button is clicked. If the input is not empty, a check is performed to determine whether the entered value contains eight characters. If the input is empty, an alert with title "Failed to Delete Coursed" is displayed. If the input contains eight characters, a check is performed to determine whether the input matches with the course code of courses saved in the registeredCourse list. If the input does not contain eight characters, an alert with title "Invalid Input" is displayed. If the input match the course code stored in the registeredCourse list, all courses in the registeredCourse list except the matching course are saved, and the value of selectedCourseToDelete is set to empty. This means the course was deleted successfully from the registeredCourse list and the selectedCoursetoDelete is ready to store another course code for deletion. If no match is found, an alert with title "Invalid Course Code / Course Not Added" is displayed.

The Use Case Description for Submit Course Registration module is shown in Table 4.3. This use case involves student as the user. The Submitting Course Registration process has a pre-requisite requirement that the registeredCourse list must not be empty. At least one course and section must be added before the "Submit" button is clicked. If the registeredCourse list is not empty, a toast is displayed to indicates the course registration is completed successfully and the registeredCourse list is set to empty, indicating the course registration is done successfully. If the registeredCourse list is empty, an alert with title "Submission Failed!" is displayed.

Table 4.1 Use Case Description for Add Course and Section

Use Case Name: Add Course and Section
Use Case ID: UC-01
Actor: Student
Description: This use case allows student to add a course and section to their registration list.

Pre-condition: The course registration page is open
Post-condition: The selected course is successfully added to the registered course list.
Trigger: The student clicks the “Add Course” button.
Main Flow: 1. The student selects a course and a section from the dropdown list. 2. The student clicks the “Add Course” button. 3. The system checks whether the selected course already exists in the registered course list. 4. The system adds the selected course to the list. 5. The system clears the selected course field. 6. The system displays the updated registered course list.
Alternative Flow: 1. If the student does not select a course or section, the system displays an alert message titled “No Course and Section Selected.” 2. If the selected course already exists in the registered course list, the system displays an alert message titled “Duplicated Registration for Same Course.”

Table 4.2 Use Case Description for Delete Course and Section

Use Case Name: Delete Course and Section
Use Case ID: UC-02
Actor: Student
Description: This use case allows student to remove a course and section from their registration list by entering the course code.
Pre-condition: The course registration page is open, and there are at least one course and section in the registered course list.
Post-condition: The selected course is successfully removed from the registered course list.
Trigger: The student clicks the “Delete Course” button.
Main Flow:

1. The student enters a course code in the input field. 2. The student clicks the “Delete Course” button. 3. The system checks whether the input is not empty. 4. The system verifies that the input is exactly 8 characters long. 5. The system checks whether the input matches any course code in the registered course list. 6. The system removes the matching course from the list. 7. The system clears the input field. 8. The system displays the updated registered course list.
Alternative Flow: 1. If the student does not enter a course code, the system displays an alert titled “Failed to Delete Course.” 2. If the entered course code is not 8 characters long, the system displays an alert titled “Invalid Input”. 3. If the entered course code does not match any course in the registered course list, the system displays an alert titled “Invalid Course Code / Course Not Added.”

Table 4.3 Use Case Description for Submit Course Registration

Use Case Name: Submit Course Registration
Use Case ID: UC-03
Actor: Student
Description: This use case allows student to submit their course registration list and complete the course registration process.
Pre-condition: The course registration page is open, and there are at least one course and section in the registered course list.
Post-condition: The registered course list is successfully submitted, and the course registration is completed.
Trigger: The student clicks the “Submit” button.

Main Flow: 1. The student clicks the “Submit” button. 2. The system checks whether the registered course list is not empty. 3. The system displays a success message confirming that the course registration is successful. 4. The system clears the registered course list.
Alternative Flow: **1.** If the registered course list is empty, the system displays an alert message titled “Submission Failed!”

4.5.4.3 Activity Diagram for Student Course Registration System

The Activity Diagram for the modules in the Student Course Registration System were described in this section.

The Activity Diagram for Add Course and Section module is shown in Figure 4.13. This activity diagram shows how student interacts with the Student Course Registration System when student is adding a course and section. The Add Course and Section process is started with course and section selection. A course and section must be selected from the dropdown list before the “Add Course” button is clicked. If a course and section was selected, a check is performed to determine whether the registeredCourse list contains course code of the selectedCourse list. If no course and section was selected, an alert with title “No Course and Section Selected” will be displayed. If the course code is not found in registeredCourse list, the selectedCourse is appended to the registeredCourse list and the value of selectedCourse is set to be empty. This means the selectedCourse is added and selectedCourse is ready to store another course and section. If the course code is already found in the registeredCourse list, an alert with title “Duplicated Registration for Same Course” is displayed.

The Activity Diagram for Delete Course and Section module is shown in Figure 4.14. This activity diagram shows how student interacts with the Student Course Registration System when student is deleting a course and section. The Delete Course and Section process is started with course code insertion. A course code must be entered before the “Delete Course” button is clicked.

If the input is not empty, a check is performed to determine whether the entered value contains eight characters. If the input is empty, an alert with title “Failed to Delete Coursed” is displayed. If the input contains eight characters, a check is performed to determine whether the input matches with the course code of courses saved in the registeredCourse list. If the input does not contain eight characters, an alert with title “Invalid Input” is displayed. If the input match the course code stored in the registeredCourse list, all courses in the registeredCourse list except the matching course are saved, and the value of selectedCourseToDelete is set to empty. This means the course was deleted successfully from the registeredCourse list and the selectedCoursetoDelete is ready to store another course code for deletion. If no match is found, an alert with title “Invalid Course Code / Course Not Added” is displayed.

The Activity Diagram for Submit Course Registration module is shown in Figure 4.15. This activity diagram shows how student interacts with the Student Course Registration System when student is submitting the course registration. The Submitting Course Registration process has a pre-requisite requirement that the registeredCourse list must not be empty. At least one course and section must be added before the “Submit” button is clicked. If the registeredCourse list is not empty, a toast is displayed to indicates the course registration is completed successfully and the registeredCourse list is set to empty, indicating the course registration is done successfully. If the registeredCourse list is empty, an alert with title “Submission Failed!” is displayed.

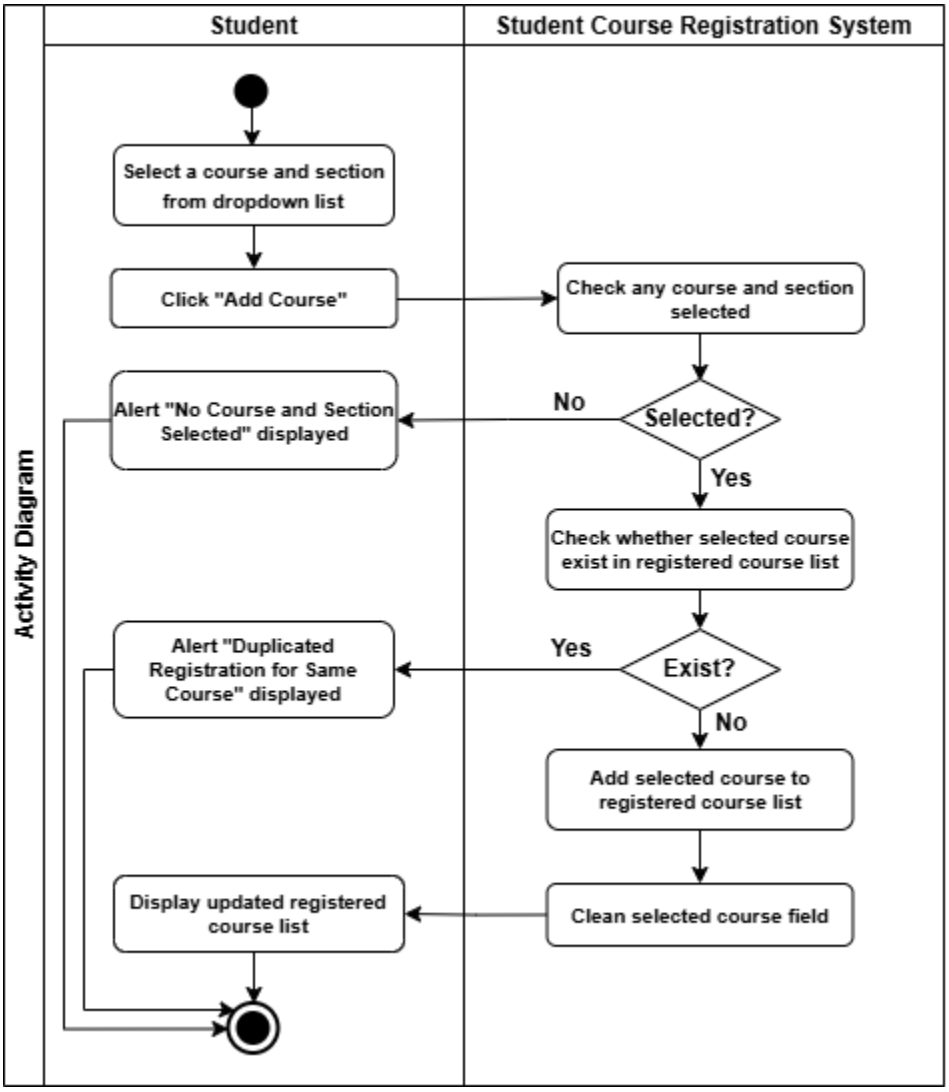


Figure 4.13 Activity Diagram for Add Course and Section

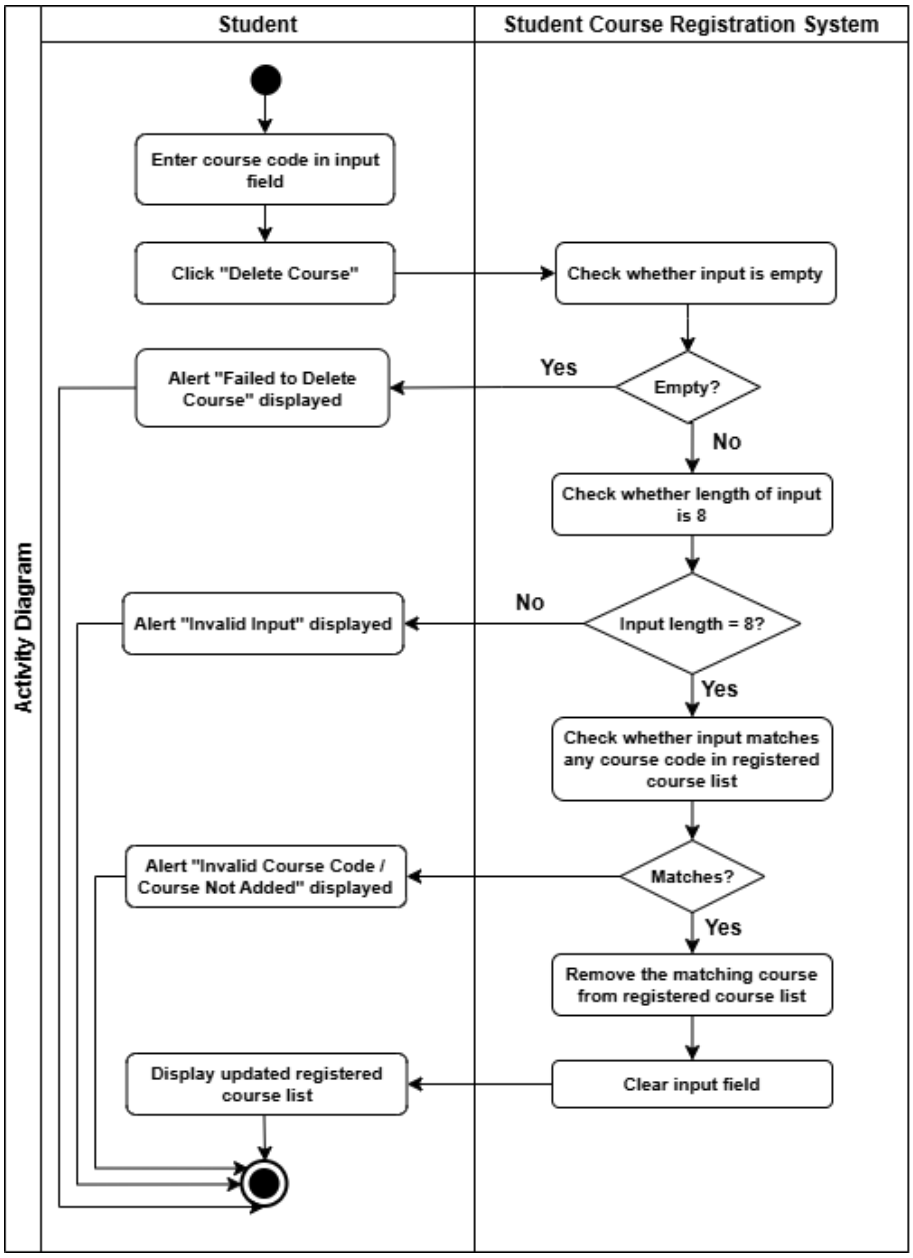


Figure 4.14 Activity Diagram for Delete Course and Section

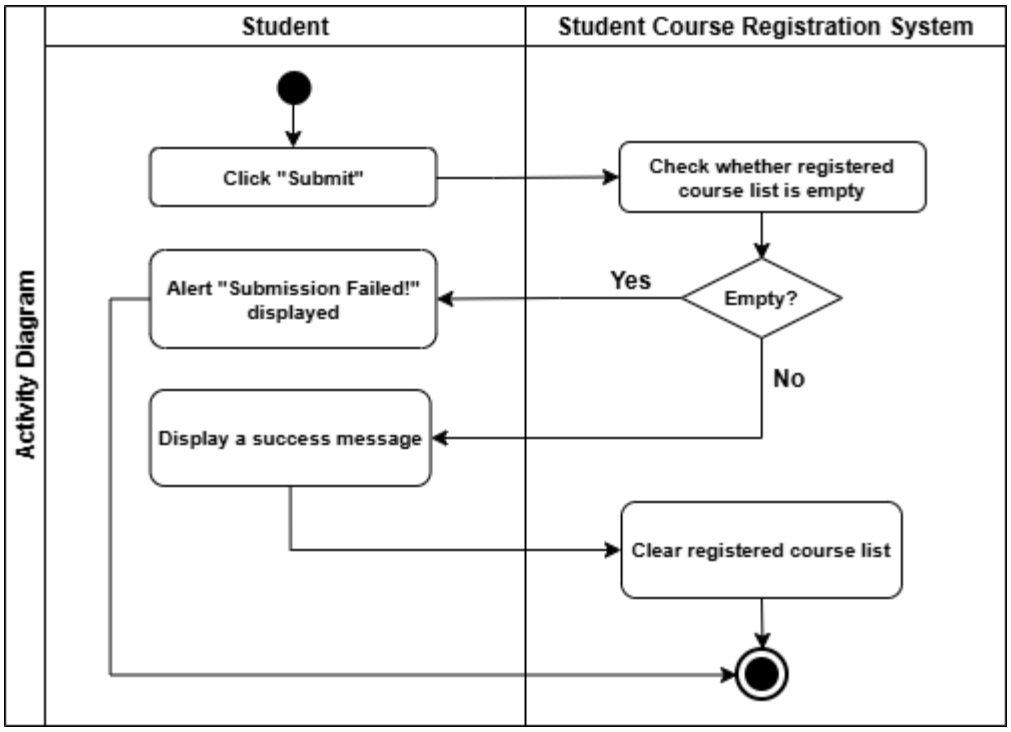


Figure 4.15 Activity Diagram for Submit Course Registration

4.5.4.4 Sequence Diagram for Student Course Registration System

The Sequence Diagram for the modules in the Student Course Registration System were described in this section.

The Sequence Diagram for Add Course and Section module is shown in Figure 4.16. This sequence diagram shows how student interacts with the Course Registration UI, Course Controller, and the registeredCourse List when student is adding a course and section. The Add Course and Section process is started with course and section selection. A course and section must be selected from the dropdown list before the "Add Course" button is clicked. If a course and section was selected, a check is performed to determine whether the registeredCourse list contains course code of the selectedCourse list. If no course and section was selected, an alert with title "No Course and Section Selected" will be displayed. If the course code is not found in registeredCourse list, the selectedCourse is appended to the registeredCourse list and the value of selectedCourse is set to be empty. This means the selectedCourse is added and selectedCourse is ready to store another course and section. If the course code is already found in the registeredCourse list, an alert with title "Duplicated Registration for Same Course" is displayed.

The Sequence Diagram for Delete Course and Section module is shown in Figure 4.17. This sequence diagram shows how student interacts with the Course Registration UI, Course Controller, and the registeredCourse List when student is deleting a course and section. The Delete Course and Section process is started with course code insertion. A course code must be entered before the "Delete Course" button is clicked. If the input is not empty, a check is performed to determine whether the entered value contains eight characters. If the input is empty, an alert with title "Failed to Delete Coursed" is displayed. If the input contains eight characters, a check is performed to determine whether the input matches with the course code of courses saved in the registeredCourse list. If the input does not contain eight characters, an alert with title "Invalid Input" is displayed. If the input match the course code stored in the registeredCourse list, all courses in the registeredCourse list except the matching course are saved, and the value of selectedCourseToDelete is set to empty. This means the course was deleted successfully from the registeredCourse list and the selectedCoursetoDelete is ready to store another course code for deletion. If no match is found, an alert with title "Invalid Course Code / Course Not Added" is displayed.

The Sequence Diagram for Submit Course Registration module is shown in Figure 4.18. This sequence diagram shows how student interacts with the Course Registration UI, Course Controller, and the registeredCourse List when student is submitting the course registration. The Submitting Course Registration process has a pre-requisite requirement that the registeredCourse list must not be empty. At least one course and section must be added before the “Submit” button is clicked. If the registeredCourse list is not empty, a toast is displayed to indicates the course registration is completed successfully and the registeredCourse list is set to empty, indicating the course registration is done successfully. If the registeredCourse list is empty, an alert with title “Submission Failed!” is displayed.

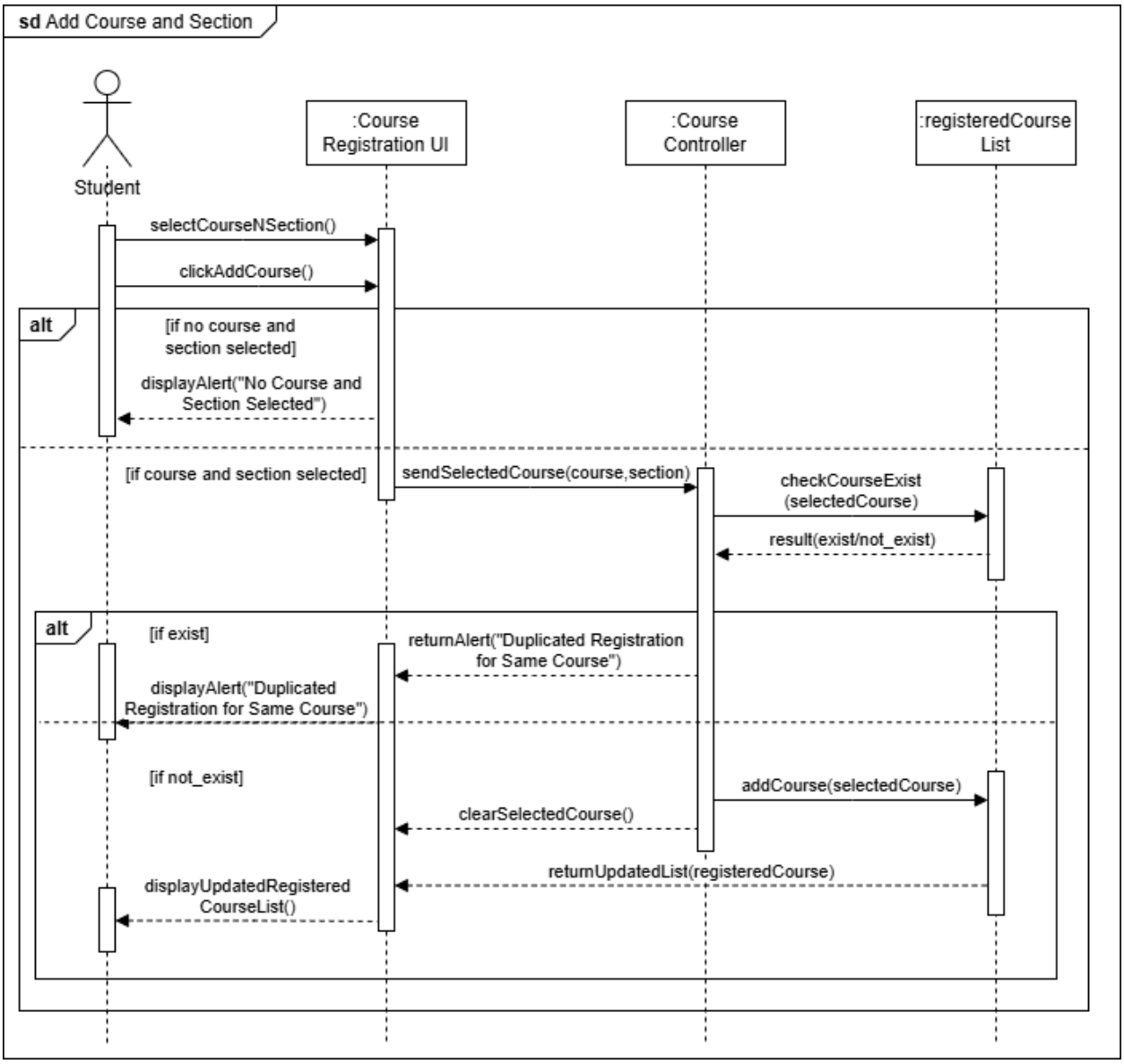


Figure 4.16 Sequence Diagram for Add Course and Section

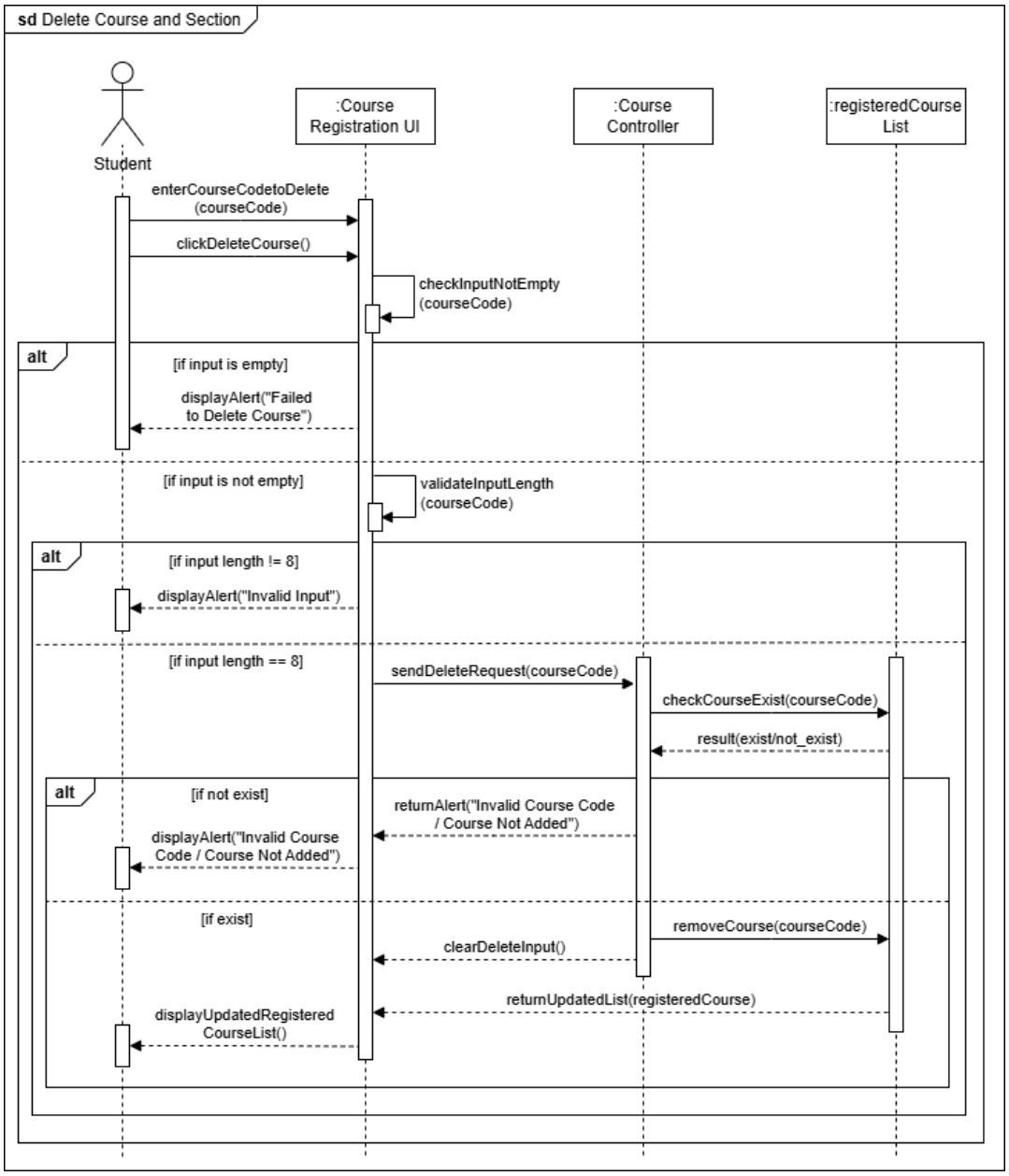


Figure 4.17 Sequence Diagram for Delete Course and Section

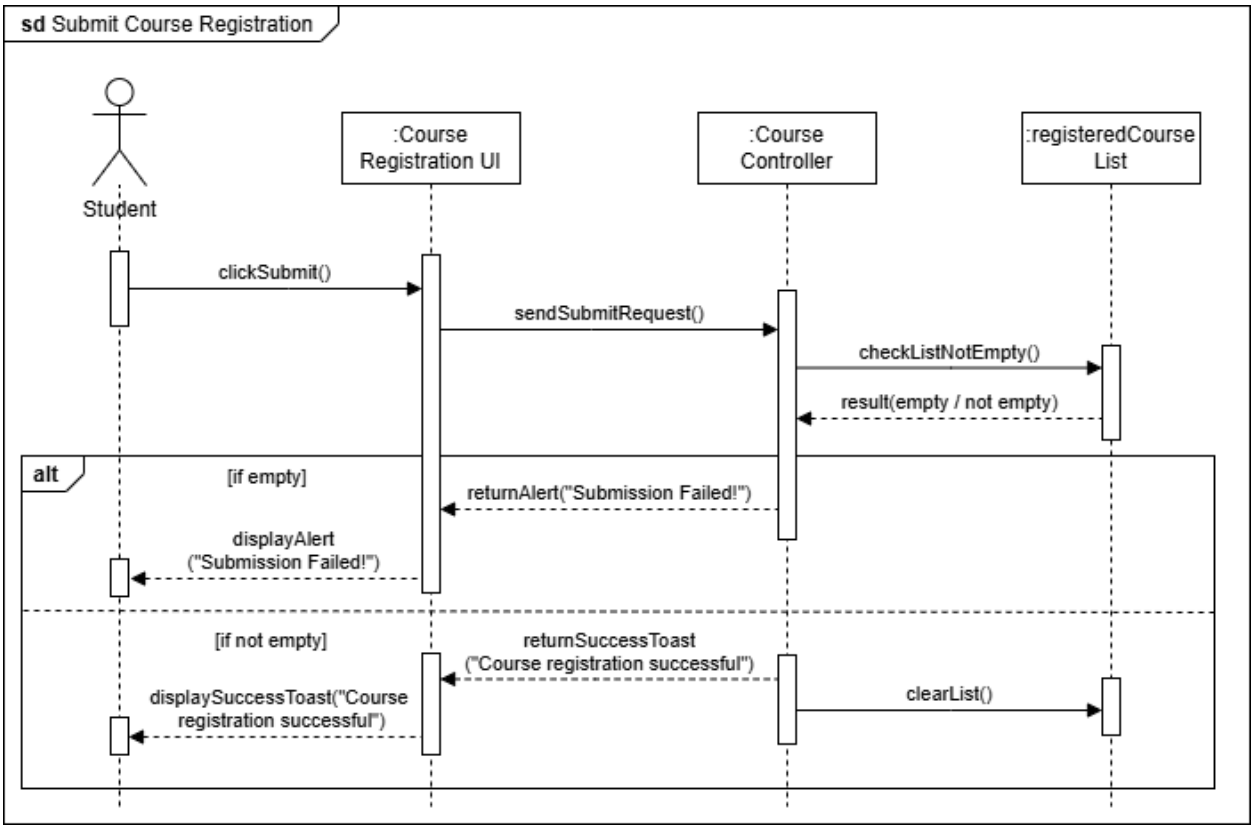


Figure 4.18 Sequence Diagram for Submit Course Registration

4.5.5 DATABASE DESIGN

A non-SQL and in-memory data structure were used in this system to store and manage the course registration data. Instead of using traditional relational database with tables, the app and page variables were used to temporarily store and process course registration data during the course registration process. The data structure of the Student Course Registration System is described in Table 4.4. The variables used in the system are registeredCourse, selectedCourse, and selectedCoursetoDelete.

Although relational database was not implemented by the system, the course registration data was still stored in an organized way. For example, each course was only allowed to be stored once in the registeredCourse list, meaning that only a section can be registered for each course. Moreover, duplicated registrations for same course and section were prohibited. As a result, clean and consistent course registration data was maintained.

Since the data was stored using the app and page variables rather than a traditional relational database, a Class Diagram was used for the database design. The Class Diagram for the Student Course Registration System is shown Figure 4.19. The Class Diagram demonstrates the relationship between the Student Registration System Page and the Student Course Registration System Application, where the Student Registration System Page uses the Student Course Registration System Application.

Table 4.4 Data Structure Used in Student Course Registration System

Variable Name	**Variable Type**	**Data Type**	**Description**
registeredCourse	App variable	list of texts	List to store added courses and sections.
selectedCourse	Page variable	text	Text to store the course and section selected from the dropdown list to be added.
selectedCoursetoDelete	Page variable	text	Text to store the course code for course to be deleted.

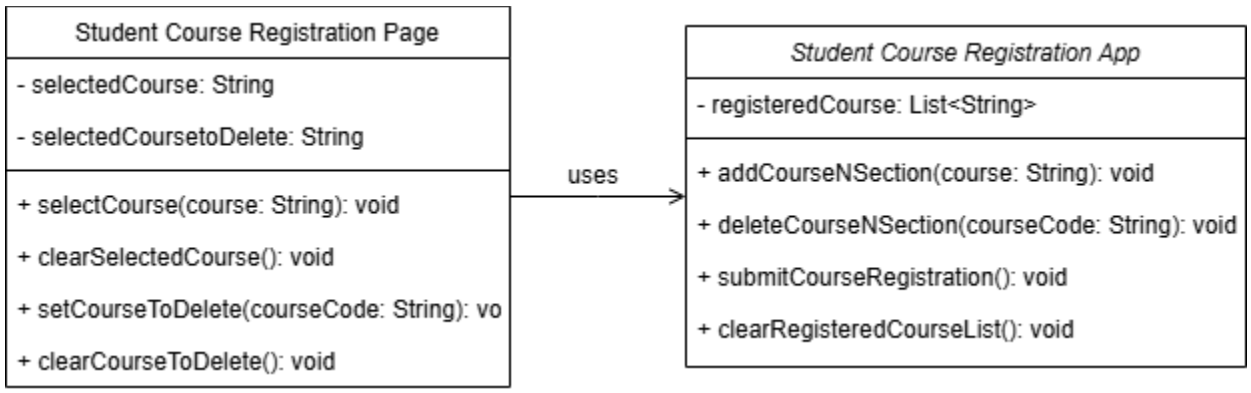


Figure 4.19 Class Diagram of Student Course Registration System

4.5.6 INTERFACE DESIGN

Figure 4.20 shows the user interface of the Student Course Registration System. Course and Section can be selected from the dropdown list. The selected course can then be added to the registeredCourse list by clicking the “Add Course” button. Besides, the course code of the course to be deleted can be entered, and the “Delete Course” button can be clicked to remove the course from the registeredCourse list. Lastly, the “Submit button” can be clicked to submit the Course Registration.

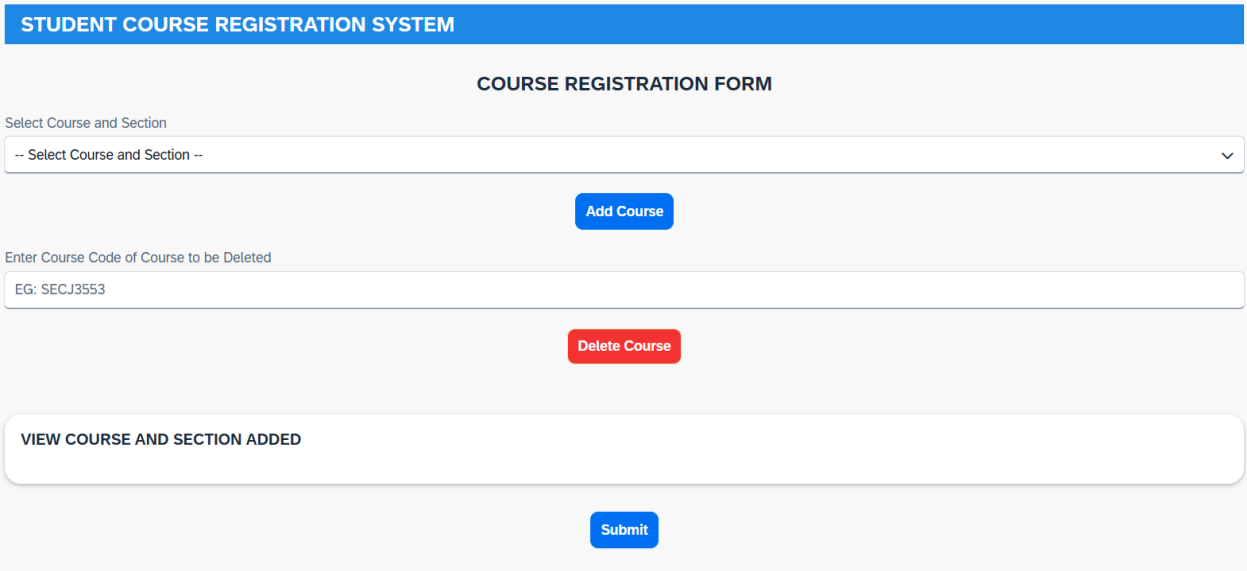


Figure 4.20 User Interface of Student Course Registration System

4.6 SUMMARY

In this chapter, a comprehensive analysis and design of the proposed Enterprise Architecture for Universiti Tun Hussein Onn (UTHM) were presented. The study began with an analysis of UTHM's organizational structure and current system environment, in which critical challenges such as data silos, manual data transfer, and reliance on legacy infrastructure like SMAP and ePayment were identified. The requirements gathering phase, which was conducted through interviews and site visits, highlighted the need for a unified platform to support the university's 20230 digital transformation vision.

Based on these findings, the existing "AS-IS" architecture was documented to identify current gaps, followed by the proposed "TO-BE" architecture. The proposed design was developed using the TOGAP framework to restructure the Business, Data, Application, and Technology layers, ensuring better alignment between IT capabilities and UTHM's strategic objectives. Key improvements include the introduction of a hybrid database model that combines SQL and NoSQL to handle diverse academic data and the implementation of a robust Implementation Governance framework to standardize future IT developments. Ultimately, this chapter establishes the architectural blueprint required to transition UTHM towards a fully integrated, cloud-enabled Smart Campus environment.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 INTRODUCTION

This chapter describes the implementation of the Student Course Registration System developed using SAP Build Apps. During the implementation phase, the system design was transformed into a working application that allows courses and sections to be added and deleted, registered courses and sections to be viewed, and the course registration form to be submitted. The chapter presents the system development process, database creation, system coding, testing activities, and a summary of the implementation outcomes.

5.2 SYSTEM DEVELOPMENT PROCESS

The system was developed using a low-code development environment using SAP Build Apps. A visual development environment was provided when building a web and mobile application.

5.2.1 DEVELOPMENT ENVIRONMENT

Table 5.1 highlights the development environment of the project. A low-code platform called SAP Build Apps is used to develop the Student Course Registration System. App and page variables are used to handle the system data, while Logic Canvas is used to implement the system workflows.

Table 5.1 Development Environment for the Student Course Registration System

Platform	SAP Build Apps
Development Type	Low-code
Data Handling	App variables and Page variables
Logic Implementation	Logic Canvas

5.2.2 IMPLEMENTATION STEPS

Multiple steps were done to develop the Student Course Registration System.

1. Requirement Analysis

The functionalities of the Student Course Registration System were identified. The functionalities including add course and section, delete course and section, view list of courses and sections added, and submit course registration form.

2. UI Design

The Course Registration Form for the system was created using the drag-and-drop feature offered by SAP Build Apps. The form consists of a dropdown list of courses and sections, and a "Add Course" button that is used to select and add a course and section from the dropdown list.

For delete course and section operation, an input field was prepared to allow the course code of the course to be entered for deletion from the list of registered courses and sections. After the course code is entered, the "Delete Course" button can be clicked to remove the course.

Besides, a real-time list of courses and sections added by students is displayed. This allows the course registration to be reviewed and confirmed before the form is submitted using the "Submit" button.

3. Data Handling

The App variables and Page variables in SAP Build Apps project were used as the data sources for data binding in the application logic and UI elements such as cards. When the "Add Course" button was clicked by the students, the selected course and section was saved as selectedCourse and was added to the list of registeredCourse.

When the “Delete Course” button was clicked by the students, the courses that does not match the selectedCoursetoDelete are saved back into the registeredCourse list, meaning that the selected course are deleted.

The registeredCourse list was displayed in a card of the user interface so that real-time changes can be viewed by students in the user interface.

4. Logic Implementation

The logic flows for “Add Course”, “Delete Course”, and “Submit” buttons were configured in this step. Only one section from a course that has not added previously is allowed to be added to the registeredCourse list to prevent duplicated course and section registration. If this rule is not followed, alerts will be displayed on the user interface.

For the delete course and section operation, the course registration must not be empty, and an eight-character course code must be entered in the text field. The course is successful removed from the registeredCourse list if the input matches a course code in the registeredCourse list. Otherwise, alert will be displayed on the user interface.

For the “Submit” operation, at least one course must be added to the registeredCourse list to prevent empty course registration.

5. Testing

Testing was done in SAP Build Apps to check the functionalities of the Student Course Registration System. The tested criteria include course and section selection from dropdown list, add course and section, delete course and section, display added courses and sections, and course registration submission.

5.3 DATABASE CREATION

Instead of using traditional relational database, the App variables and Page variables of SAP Build Apps project were used to store data of the course registration processes in the Student Course

Registration System. The purpose and example of the App and Page variable implemented in the system are explained in Table 5.2. The main data storage was the registeredCourse list, which acted as the temporary database for storing all courses selected by the student. While the selectedCourse and selectedCoursetoDelete page variable were used to hold user input temporarily for adding and deleting course purpose.

Table 5.2 App and Page Variables Used in Course Registration System

Variable Name	Variable Type	Data Type	Description	Example
registeredCourse	App variable	list of texts	List to store added courses and sections.	UHLF1112 FRENCH LANGUAGE SECTION 01
selectedCourse	Page variable	text	Text to store the course and section selected from the dropdown list to be added.	SECJ3553 ARTIFICIAL INTELLIGENCE SECTION 02
selectedCoursetoDelete	Page variable	text	Text to store the course code for course to be deleted.	SECJ3553

5.4 SYSTEM CODING

Traditional programming coding was replaced with logic flows because SAP Build Apps, a low-code/no-code development platform was used for the development process. The main functions of Student Course Registration System module including Add Course and Section module, Delete Course and Section module, and Submitting Course Registration module.

5.4.1 Add Course and Section

The input, process, and output for the Add Course and Section function were explained in this section. The formulas used while configuring the logic flow of "Add Course" button is shown in Table 5.3. The list of available courses and sections to be added (input) is shown in Figure 5.1. The Logic Canvas of the "Add Course" button (process) is shown in Figure 5.2. The example list of courses and sections added by a student (output) is shown in Figure 5.3.

The Add Course and Section process is started with course and section selection. A course and section must be selected from the dropdown list before the "Add Course" button is clicked. If a course and section was selected, a check is performed to determine whether the registeredCourse list contains course code of the selectedCourse list. If no course and section was selected, an alert with title "No Course and Section Selected" will be displayed. If the course code is not found in registeredCourse list, the selectedCourse is appended to the registeredCourse list and the value of selectedCourse is set to be empty. This means the selectedCourse is added and selectedCourse is ready to store another course and section. If the course code is already found in the registeredCourse list, an alert with title "Duplicated Registration for Same Course" is displayed.

Table 5.3 Formula Used in Logic Canvas of "Add Course" Button

Formula	Explanation
If condition: NOT(IS_EMPTY(pageVars.selectedCourse))	To check if the selectedCourse page variable is not empty. This means a course and section must be selected from the dropdown list.
If condition: NOT(CONTAINS(JOIN(appVars.registeredCourse, ", "),SPLIT(pageVars.selectedCourse, " ")[0]))	To check if the registeredCourse list app variable does not contain the first part of the value of the selectedCourse page variable (course code). This means only one section is allowed for each added course.

Set app variable: WITH_UNIQUE_ITEM(appVars.registeredCourse, pageVars.selectedCourse)	To append unique value of selectedCourse page variable to the registeredCourse list app variable. This means adding the newly selected course to the selected course list.

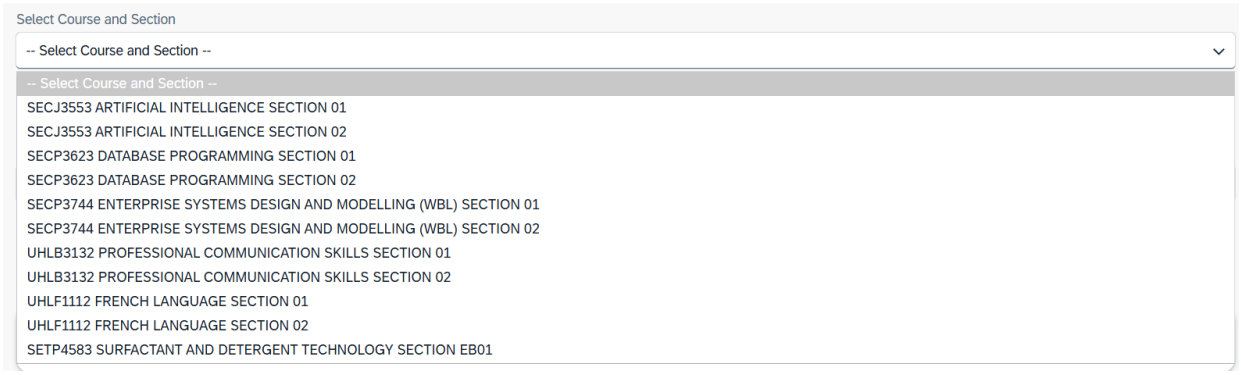


Figure 5.1 List of Courses and Sections Offered for Course Registration

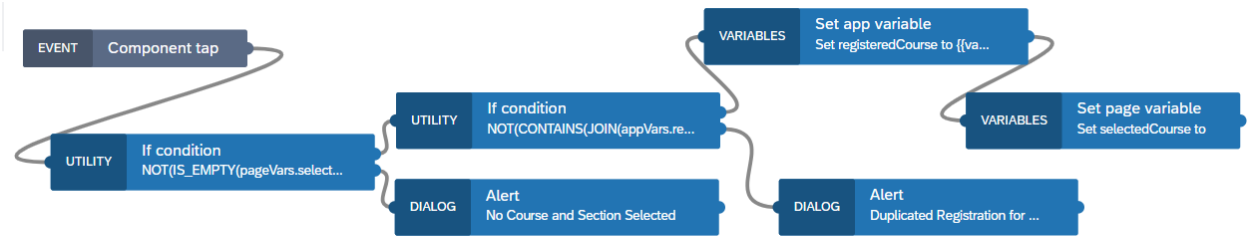


Figure 5.2 Logic Canvas of "Add Course" Button

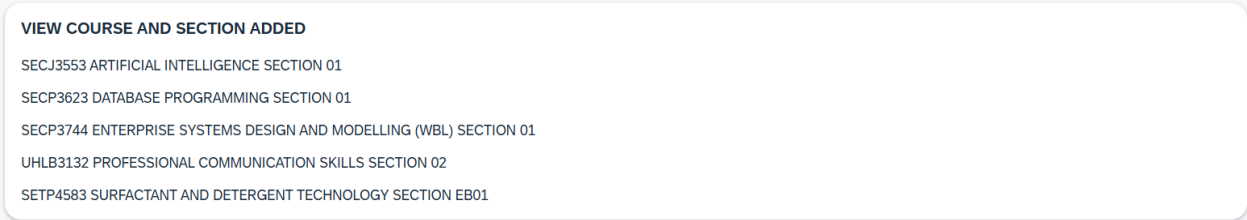


Figure 5.3 Example List of Courses and Sections Added by a Student

5.4.2 Delete Course and Section

The input, process, and output for the Delete Course and Section module were explained in this part. The formulas used while configuring the logic flow of "Delete Course" button is explained in Table 5.4. The example course code entered by students to delete course that they want to delete (input) is shown in Figure 5.4. The Logic Canvas of the "Delete Course" button (process) is shown in Figure 5.5. The example list of courses and sections added by a student after deleting a course from the list (output) is shown in Figure 5.6.

The Delete Course and Section process is started with course code insertion. A course code must be entered before the "Delete Course" button is clicked. If the input is not empty, a check is performed to determine whether the entered value contains eight characters. If the input is empty, an alert with title "Failed to Delete Coursed" is displayed. If the input contains eight characters, a check is performed to determine whether the input matches with the course code of courses saved in the registeredCourse list. If the input does not contain eight characters, an alert with title "Invalid Input" is displayed. If the input match the course code stored in the registeredCourse list, all courses in the registeredCourse list except the matching course are saved, and the value of selectedCourseToDelete is set to empty. This means the course was deleted successfully from the registeredCourse list and the selectedCoursetoDelete is ready to store another course code for deletion. If no match is found, an alert with title "Invalid Course Code / Course Not Added" is displayed.

Table 5.4 Formula Used in Logic Canvas of "Delete Course" Button

Formula	Explanation
If condition: NOT(IS_EMPTY(pageVars.selectedCoursetoDelete))	To check if the selectedCoursetoDelete page variable is not empty. This means a course code must entered by students.
If condition: LENGTH(pageVars.selectedCoursetoDelete)==8	To check if the length of the value of the selectedCoursetoDelete page variable is 8.

If condition: CONTAINS(JOIN(appVars.registeredCourse,",",")",pageVars.selectedCoursetoDelete)	To check if the registeredCourse list app variable contains the value of the selectedCoursetoDelete page variable (course code). This means the value of selectedCoursetoDelete page variable must match with the course code of courses added in the registeredCourse list app variable.
Set app variable: SELECT(appVars.registeredCourse, SPLIT(item,")[0] != pageVars.selectedCoursetoDelete)	To save all added courses as registeredCourse app variable expect the course that match with the value of the selectedCoursetoDelete page variable. This means saving all added courses in registeredCourse app variable except the course which course code is entered by students in the text field.

Figure 5.4 Example Course Code Entered by Students to Delete Course That They Want to Delete

Figure 5.5 Logic Canvas of “Delete Course” Button

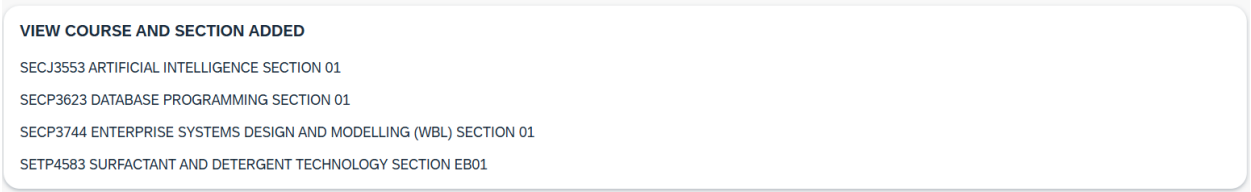


Figure 5.6 Example List of Courses and Sections Added by a Student After Deleting a Course from the List

5.4.3 Submit Course Registration

The input, process, and output for the Submit Course Registration module were explained in this section. The formulas used while configuring the logic flow of Submit Course Registration is explained in Table 5.5. The example courses and sections added by student (input) is shown in Figure 5.7. The Logic Canvas of the Submit Course Registration (process) is show in Figure 5.8. The toast display after successfully submit course registration (output) is shown in Figure 5.9.

The Submitting Course Registration process has a pre-requisite requirement that the registeredCourse list must not be empty. At least one course and section must be added before the "Submit" button is clicked. If the registeredCourse list is not empty, a toast is displayed to indicates the course registration is completed successfully and the registeredCourse list is set to empty, indicating the course registration is done successfully. If the registeredCourse list is empty, an alert with title "Submission Failed!" is displayed.

Table 5.5 Formula Used in Logic Canvas of Submit Course Registration

Formula	Explanation
If condition: NOT(IS_EMPTY(appVars.registeredCourse))	To check if the registeredCourse app variable is not empty. This means there must be at least one course and section added by students.

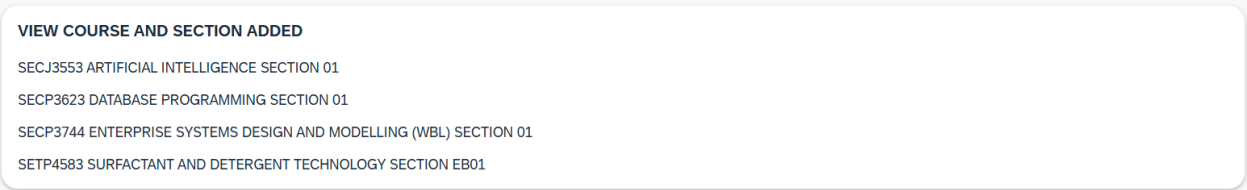


Figure 5.7 Example Courses and Sections Added by Student

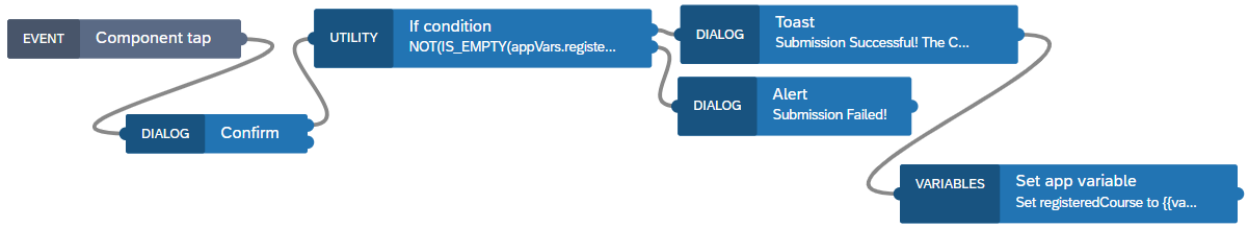


Figure 5.8 Logic Canvas of Submit Course Registration

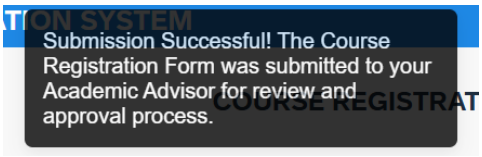


Figure 5.9 Toast Display After Successfully Submit Course Registration

5.5 SUMMARY

This chapter shows the implementation of the Student Course Registration System using SAP Build Apps. The system was developed using a low–code development environment, which enables fast application development with less coding knowledge. In this chapter, data binding and data retrieval were performed using the SAP Build Apps project App and Page variables. The main modules of this module, which are Adding Course and Section, Delete Course and Section, and Submit Course Registration, were implemented through logic flows configuration using the Logic Canvas. In conclusion, the success of this application proved that SAP Build Apps was a useful and effective platform for developing enterprise systems.

CHAPTER 6
CONCLUSION

6.1 INTRODUCTION

This chapter provides a comprehensive review of the developed system, focusing on the evaluation of its functional performance, and its overall impact on student course registration. The specific roles played by each module in streamlining student course registration and administrative are examined to demonstrate the project's practical utility. Additionally, the technical constraints identified during the implementation phase are documented to provide a realistic assessment of the system's current capabilities within the SAP Build environment. The chapter concludes with strategic suggestions for future enhancements aimed at improving system integration and automation.

6.2 SYSTEM CONTRIBUTION

This project consists of seven modules, which are Student Course Registration module, Course Management module, Section Scheduling module, Admin Dashboard module, Manage Student Profile module, Course Approval module, and View Student List for each Section module.

The Student Course Registration module is used to allow courses and sections to be selected and registered online. Besides, courses and sections can be deleted, and a real-time list of registered courses and sections can be viewed. The errors are minimized because real-time registered courses and sections can be viewed and confirmed before the course registration is submitted.

The Section Scheduling module is used to manage the venue and class time of each section. It is ensured that no sections clash with the class time or venue of other sections. The classroom usage is optimized and schedule clashes due to human error are prevented.

The Course Management module is used to manage course information. In this module, course can be added and managed by admins. Course management includes changes to the course code, course name, and credit hour. Moreover, courses can be deleted too.

The Admin Dashboard module is used to allow administrators to monitor and manage the course registration system. Registration statistics and analytics can be viewed on the main Dashboard page. For pending registrations, administrators can view the list of students awaiting Academic Advisor approval filtered by course. System settings such as registration period, credit limits, and notification preferences can be configured through the Settings page. Activity Logs are available to track all actions performed in the system.

The View Student List for Each Course module is used to allow instructors and administrators to view a full list of every student registered in a certain course section. Student enrolment can be monitored more efficiently by lecturers and administrators. Moreover, manual record keeping is reduced, and enrolment data accuracy is increased.

The Course Approval module is used to allow registered courses to be reviewed and approved by academic advisers. It is ensured that students are registered for courses that align with their schedules. By guaranteeing that course registrations are properly approved, academic control is maintained, and registration errors are minimized.

The Student Profile module is used to collect all student data including personal and academic information into one subsystem. Moreover, access to student data is supported for administrative purposes when needed. Student monitoring is improved, errors are minimized, and the redundant data entry is reduced.

6.3 SYSTEM CONSTRAINTS

While the proposed architecture introduces significant operational improvements, several technical constraints were identified within the current design phase. A primary technical limitation is the system's dependency on stable internet connectivity. Since the solution is architected using the cloud-native capabilities of SAP Build Apps and remote NoSQL data storage, continuous network access is required for functionality. Consequently, latency or temporary inaccessibility to real-time scheduling data may be experienced in environments with limited bandwidth.

Furthermore, the integration with the university's existing legacy infrastructure presents a notable challenge. The transition from traditional relational databases –such as those used in

SMAP and ePayment –to a modern NoSQL architecture necessity of complex data synchronization. Currently, this interoperability is proposed to be achieved through API bridges, which may impose limitations regarding data transfer speeds and format compatibility during the interim migration period.

Finally, regarding the development environment, the use of a Low-Code Development Platform (LCDP) was selected to facilitate rapid prototyping and proposal visualization. While this approach effectively demonstrates the system's core value and feasibility, the implementation of highly complex, customer algorithms –such as AI-driven automated conflict resolution –is bounded by the pre-defined flow functions available within the platform.

6.4 FUTURE SUGGESTIONS

The project has considerable amounts of opportunity to grow in the future, going from a number of individual modules to an integrated, professional system. The complete integration of the seven separate elements into a single ecosystem is the main plan for future development. Data input in the "Manage Student Profile" module does not now automatically appear in the "Student Course Registration" or "View Student List" modules due to the modules' current isolation. Through the use of SAP Business Technology Platform (BTP) to develop a centralized data backend, the system may provide a single "source of truth." This would eliminate data duplication and the possibility of human input mistakes by ensuring that every change is made in one module. For example, a student changing their credit hours is reflected across the whole system in real time.

The next suggestion is the implementation of process simplification and workflow automation. The system can be configured to have automated processes using technologies like SAP Build Process Automation. For instance, a submission in the "Student Course Registration" module could instantly alert a lecturer in the "Course Approval" module. This greatly reduces the workload for administrators by establishing a proactive system as opposed to a reactive one. Moreover, security requires the implementation of Role-Based Access Control (RBAC). This would guarantee that just registration and profile tools are visible to students, and administration

modules such as "Section Scheduling" and "Section Allocation" are only accessible by those with authorization.

6.5 SUMMARY

In summary, the development of the automated course registration system successfully demonstrates the feasibility of using low-code technology to improve course registration workflows. While the system provides significant benefits in terms of data accuracy and communication transparency, it remains subject to technical dependencies such as internet connectivity and platform specific logic boundaries. The identified constraints and future suggestions provide a roadmap for transitioning this prototype into a more comprehensive, centralized ecosystem. Ultimately, the project fulfils its primary objective of modernizing the course registration process through automated approvals and secure data management.

6.6 REFLECTION

From this project, valuable experience was gained in designing enterprise architecture and developing a web application using low code/no code platform. By using the TOGAF framework, an understanding was developed on how to design an enterprise architecture that aligns the system functions with the organizational needs. This helped in understanding how business processes, data, applications, and technologies are structured within in a system.

A Student Course Registration System consisting of the Add Course and Section module, Delete Course and Section module, and Submit Course Registration module was developed using the SAP Build Apps. Through this platform, the design of user interface, data binding, and system logic configuration was explored using the drag-and-drop and low-code features provided by the SAP Build Apps.

Furthermore, writing the project report in thesis formatting increased the familiarity with the thesis formatting, which will be required for the final year project report. For example, it was learned that every table and figure must be labelled and explained in text. In this way, the report was prepared in a more organized way by following the required guidelines.

In summary, opportunities were provided to explore SAP Build Apps and to improve the thesis writing skills.

REFERENCES

Administrator, U.W. (no date a) *Official portal of UTHM Information Technology Centre*. https://ptm.uthm.edu.my/.

Agnes, M., Jola, L. and Gaspersz, S. (2018) 'Academic Information System for Student (Case Study: Victory University of Sorong),' *International Journal of Computer Applications*, 180(43), pp. 26–33. https://doi.org/10.5120/ijca2018917134.

Amin, F.A. *et al.* (2024) 'Cultivating excellence: a case study of enterprise architecture transformational journey in higher education,' *Indonesian Journal of Electrical Engineering and Computer Science*, 34(1), p. 548. https://doi.org/10.11591/ijeecs.v34.i1.pp548-555.

Astri, L.Y. and Gaol, F.L. (2013) 'INFORMATION SYSTEM STRATEGIC PLANNING WITH ENTERPRISE ARCHITECTURE PLANNING,' *CommIT (Communication and Information Technology) Journal*, 7(1), p. 23. https://doi.org/10.21512/commit.v7i1.580.

GeeksforGeeks (2026) *Waterfall Model Software Engineering*. https://www.geeksforgeeks.org/software-engineering/waterfall-model/.

Hashim, N.M.Z. (2024) 'Development of student information system,' *Utem* [Preprint]. https://www.academia.edu/125256655/Development_of_Student_Information_System.

Hindarto, D. (2023b) 'Supporting University Management System Digital Transformation with Enterprise Architecture,' *Jurnal JTIK (Jurnal Teknologi Informasi Dan Komunikasi)*, 7(4), pp. 696–706. https://doi.org/10.35870/jtik.v7i4.1830.

KAUST Transforms Campus to Smart City with SAP Business Technology Platform (no date). https://apphaus.sap.com/project/kaust-transforms-campus-to-smart-city-with-sap-business-technology-platform.

Lamey, A. *et al.* (2023) 'A realistic and practical guide for creating intelligent integrated solutions in higher education using enterprise architecture,' *Sustainability*, 15(11), p. 8780. https://doi.org/10.3390/su15118780.

Lankhorst, M. (2017) *Enterprise architecture at work*, *The enterprise engineering series*. https://doi.org/10.1007/978-3-662-53933-0.

Meutia, N.S. *et al.* (2022) 'Enterprise Architecture Framework in Higher Education: Systematic Literature Review,' *Applied Technology and Computing Science Journal*, 5(2), pp. 33–39. https://doi.org/10.33086/atcsj.v5i2.3751.

Nama, G.F., Tristiyanto and Kurniawan, D. (2017) 'An Enterprise Architecture Planning for higher education using the Open Group Architecture Framework (TOGAF): Case Study University of Lampung,' *2017 Second International Conference on Informatics and Computing (ICIC)*, pp. 1–6. https://doi.org/10.1109/iac.2017.8280610.

Navaz, A.S.S. (2013) 'Human Resource Management System,' *IOSR Journal of Computer Engineering*, 8(4), pp. 62–71. https://doi.org/10.9790/0661-0846271.

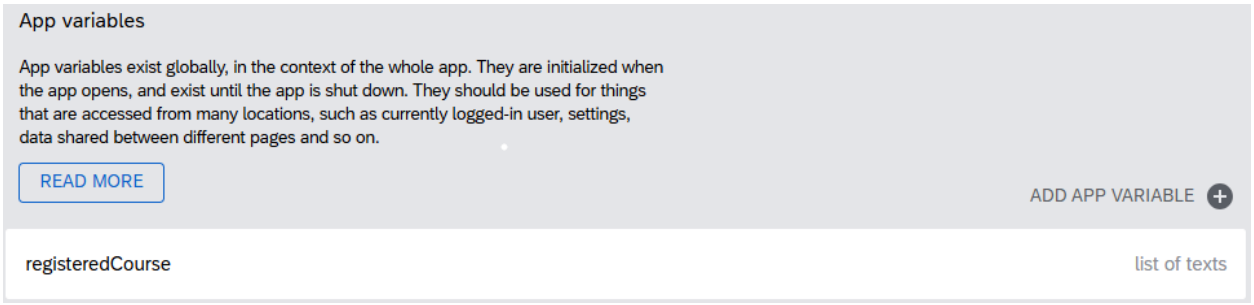
NYU Operating Model Diagram (no date). https://www.nyu.edu/life/information-technology/about-nyu-it/enterprise-architecture/enterprise-architecture--nyu-operating-model-diagram.html?challenge=d06e90d7-4d8f-4b88-9d8c-10b73beb60f1.

Pedroso, M.S. (2021) *Enterprise Architecture in the Higher Education sector - A Case Study*. thesis. https://repositorio.iscte-iul.pt/bitstream/10071/24029/1/master_miguel_salvador_pedroso.pdf.

Setiawan, R. (2018) 'Architecture of human resource management system at universities,' *IOP Conference Series Materials Science and Engineering*, 434, p. 012258. https://doi.org/10.1088/1757-899x/434/1/012258.

The Open Group (2018) *The TOGAF Standard, Version 9.2*. The Open Group. https://governance.foundation/assets/frameworks/togaf/c182e%20-%20TOGAF%209.2.pdf.

APPENDIX A App Variable Used in Student Course Registration System



APPENDIX B Page Variable Used in Student Course Registration System